# rapport_adaptation_domaine_corrige

May 15, 2026

## 1 Introduction générale

En apprentissage supervisé classique, on suppose généralement que les données d'entraînement et les données de test suivent la même distribution. Autrement dit, le modèle est appris sur un domaine source et évalué sur un domaine qui lui ressemble statistiquement.

Cette hypothèse est pourtant souvent trop forte en pratique. Un modèle entraîné sur des données simulées peut être utilisé sur des données réelles ; un classifieur appris sur des images prises dans un certain environnement peut être testé dans un autre ; un modèle construit sur une population peut être appliqué à une population différente. Dans toutes ces situations, la distribution d'entraînement et la distribution de test ne coïncident plus exactement.

L'objectif de ce rapport est d'étudier ce problème sous l'angle de l'adaptation de domaine. On considère deux domaines :

$$\mathcal{D}_S = (D_S, f_S) \qquad \text{et} \qquad \mathcal{D}_T = (D_T, f_T),$$

où $D_S$ et $D_T$ désignent les distributions source et cible, tandis que $f_S$ et $f_T$ désignent les fonctions d'étiquetage associées.

La question centrale est la suivante :

> peut-on contrôler l'erreur target d'un modèle appris principalement sur le domaine source ?

Pour répondre à cette question, le rapport suit une progression en trois niveaux.

D'abord, on construit un cadre expérimental simple permettant de générer deux domaines distincts et d'observer concrètement l'effet d'un décalage entre source et cible. Ensuite, on introduit progressivement les outils théoriques nécessaires : erreur source, erreur target, divergence entre distributions, $\mathcal{H}$-divergence, $\mathcal{H}\Delta\mathcal{H}$-divergence et hypothèse jointe idéale. Enfin, on relie ces quantités à des expériences numériques afin d'illustrer le rôle de chaque terme dans les bornes de généralisation.

Le but n'est donc pas seulement d'appliquer un algorithme, mais de comprendre mathématiquement pourquoi et quand un transfert du domaine source vers le domaine cible peut fonctionner.

## 2 Table des notations

Dans ce rapport, on utilise les notations suivantes.

## 3 Table des notations

Dans ce rapport, on utilise les notations suivantes.

- $\mathcal{X}$ : espace d'entrée des observations.
- $D_S$ : distribution source sur $\mathcal{X}$.
- $D_T$ : distribution cible sur $\mathcal{X}$.
- $f_S$ : fonction d'étiquetage du domaine source.
- $f_T$ : fonction d'étiquetage du domaine cible.
- $\mathcal{D}_S = (D_S, f_S)$ : domaine source complet.
- $\mathcal{D}_T = (D_T, f_T)$ : domaine cible complet.
- $\mathcal{H}$ : classe d'hypothèses considérée.
- $h \in \mathcal{H}$ : hypothèse ou classifieur appris.
- $h_S$ : hypothèse apprise principalement sur le domaine source.
- $h^*$ : hypothèse jointe idéale minimisant une erreur source et cible.
- $\varepsilon_S(h)$ : erreur de $h$ sur le domaine source.
- $\varepsilon_T(h)$ : erreur de $h$ sur le domaine cible.
- $\varepsilon_S(h, h')$ : probabilité de désaccord entre $h$ et $h'$ sous $D_S$.
- $\varepsilon_T(h, h')$ : probabilité de désaccord entre $h$ et $h'$ sous $D_T$.
- $d_1(D_S, D_T)$ : distance entre les densités source et cible.
- $d_{\mathcal{H}}(D_S, D_T)$ : $\mathcal{H}$-divergence entre les distributions source et cible.
- $d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T)$ : divergence fondée sur les désaccords entre hypothèses.
- $\lambda$ : erreur jointe minimale sur les domaines source et cible.
- $m$ : taille des échantillons utilisés.
- $\delta$ : niveau de probabilité d'échec dans les bornes de concentration.

Cette table fixe les notations utilisées dans la suite. Elle est importante car les résultats théoriques reposent sur des distinctions précises : erreur par rapport à une vraie fonction d'étiquetage, désaccord entre deux hypothèses, divergence entre distributions et estimation empirique de ces quantités.

## 4 Partie I — Génération des données et définition des domaines

En adaptation de domaine, un domaine est défini par une distribution sur l'espace des observations et par une fonction d'étiquetage :

$$\mathcal{D} = (D, f).$$

Nous construisons donc deux domaines :

$$\mathcal{D}_S = (D_S, f_S), \qquad \mathcal{D}_T = (D_T, f_T).$$

La tâche synthétique est une classification binaire du risque client. Chaque observation est :

$$x = (x_1, x_2) \in \mathbb{R}^2, \qquad y \in \{0, 1\}.$$

Ici, $x_1$ représente un score d'activité, $x_2$ un score de stabilité, $y = 0$ un risque faible et $y = 1$ un risque élevé.

[1]:
```
from dataclasses import dataclass
from typing import Callable, Tuple

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (7, 5)
plt.rcParams["axes.grid"] = True
```

CETTE PREMIERE PARTIE PREPARE LE BLOC EXPERIMENTALdataclass p

### 4.1 I.1 — Définition : structure d'un domaine et bruit d'étiquetage

Pour éviter toute ambiguïté, on distingue trois objets :

1. un **score réel** $s(x)$ ;
2. une **probabilité d'étiquetage** $g(x)$ ;
3. une **fonction d'étiquetage déterministe** $f(x) \in \{0, 1\}$.

s(x) : C'est un score brut. Il mesure :
à quel point un point ressemble à la classe positive ;
sa "distance" à la frontière de décision.

on utilise pas directement un label car un score permet :-de modéliser l'incertitude ;- d'introduire du bruit ;-d'obtenir une probabilité

On définit d'abord :

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

La sigmoïde transforme :R[0,1] Donc :un s

Puis :

$$g(x) = \mathbb{P}(Y = 1 \mid X = x) = \sigma\left(\frac{s(x)}{\tau}\right),$$

Si est petit, on a un comportement presque

où $\tau$ contrôle le niveau de bruit.

La fonction d'étiquetage déterministe associée est :

$$f(x) = \mathbf{1}\left[g(x) \geq \frac{1}{2}\right].$$

Fonction d'etiquetage deterministe:
On transforme une probabilité en label binaire

Enfin, les labels observés dans l'échantillon sont bruités :

$$Y \mid X = x \sim \text{Bernoulli}(g(x)).$$

on distingue g(x) et f(x) car g(x) modélise :
le vrai processus probabiliste et f(x) représente la règle de décision idéale.

Ainsi, dans le code :

- les fonctions `source_score` et `target_score` renvoient les scores $s_S(x)$ et $s_T(x)$ ;
- les fonctions `source_label_probability` et `target_label_probability` renvoient les probabilités $g_S(x)$ et $g_T(x)$ ;
- les fonctions `true_source_label` et `true_target_label` renvoient les labels déterministes $f_S(x)$ et $f_T(x)$, donc uniquement 0 ou 1.

[2]:
```
@dataclass
class DomainData:
    X: np.ndarray
```

```
    y: np.ndarray
    name: str


def sigmoid(z: np.ndarray) -> np.ndarray:
    return 1.0 / (1.0 + np.exp(-z))
```

La distribution source Ds signifie que 50% des points viennent d'une premiere gaussienne et 50% de l'autre. Ce choix est très pertinent expérimentale ment car Une seule gaussienne donnerait un problème trop simple. Avec un mélange :les données forment plusieurs groupes, la frontière de décision devient plus intéressante

### 4.2 I.2 — Définition du domaine source $\mathcal{D}_S = (D_S, f_S)$

La distribution source est un mélange équilibré de deux gaussiennes :

$$D_S = \frac{1}{2}\mathcal{N}(\mu_1, \Sigma_1) + \frac{1}{2}\mathcal{N}(\mu_2, \Sigma_2).$$

On définit d'abord le score source :

$$s_S(x) = x_1 + 0.8x_2 + 0.5\sin(2x_1).$$

Le domaine target sera ensuite obtenu par transformation géométrique : rotation, translation, changement d'échelle. Cela crée un vrai décalage entre : Ds != Dt

La probabilité d'étiquetage source est :

$$g_S(x) = \mathbb{P}_S(Y = 1 \mid X = x) = \sigma\left(\frac{s_S(x)}{\tau}\right).$$

La fonction d'étiquetage déterministe source est alors :

$$f_S(x) = \mathbf{1}\left[g_S(x) \geq \frac{1}{2}\right].$$

Les labels observés sont bruités selon :

$$Y_S \mid X = x \sim \text{Bernoulli}(g_S(x)).$$

[3]:
```
def generate_source_features(n_samples: int, rng: np.random.Generator) -> np.
↪ndarray:
    sizes = [n_samples // 2, n_samples - n_samples // 2]
    means = [np.array([-1.5, -0.5]), np.array([1.5, 0.8])]
    covariances = [
        np.array([[0.8, 0.2], [0.2, 0.6]]),
        np.array([[0.7, -0.1], [-0.1, 0.7]]),
    ]
    X = np.vstack([
        rng.multivariate_normal(mean, covariance, size=size)
        for mean, covariance, size in zip(means, covariances, sizes)
    ])
    rng.shuffle(X)
    return X
```

Cette fonction génère les données Xs

On partage les données en deux groupes

Ce sont les centres des deux gaussiennes

Matrices de covariance

rng.multivariate_normal(...) Génère des points selon N(,); zip(...) associe moyenne, co-varianc

```python
def source_score(X: np.ndarray) -> np.ndarray:
    x1, x2 = X[:, 0], X[:, 1]
    return x1 + 0.8 * x2 + 0.5 * np.sin(2.0 * x1)


def source_label_probability(X: np.ndarray, noise_level: float) -> np.ndarray:
    return sigmoid(source_score(X) / noise_level)
```

score

Probabilité d'étiquetage

Génération des labels

```python
def source_label_function(
    X: np.ndarray,
    noise_level: float,
    rng: np.random.Generator,
) -> np.ndarray:
    return rng.binomial(n=1, p=source_label_probability(X, noise_level))
```

Pour chaque point : on calcule une probabilité, puis on tire aléatoirement : 1 avec probabilité gs (x), 0 sinon.

### 4.3 I.3 — Définition du domaine target $\mathcal{D}_T = (D_T, f_T)$

Le domaine target est obtenu par transformation géométrique du domaine source :

$$X_T = XAR^\top + b,$$

avec :

$$A = \begin{pmatrix} 1.2 & 0 \\ 0 & 0.8 \end{pmatrix}, \qquad R(\theta) = \begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix},$$

$\theta = 0.45$ et $b = (0.8, -0.4)$. Ainsi, $D_S \neq D_T$.

On définit le score target :

$$s_T(x) = 0.9x_1 + x_2 + 0.5\sin(2x_1 + 0.4).$$

La probabilité d'étiquetage target est :

$$g_T(x) = \mathbb{P}_T(Y = 1 \mid X = x) = \sigma\left(\frac{s_T(x)}{\tau}\right).$$

La fonction d'étiquetage déterministe target est :

$$f_T(x) = \mathbf{1}\left[g_T(x) \geq \frac{1}{2}\right].$$

Les labels observés sont bruités selon :

$$Y_T \mid X = x \sim \text{Bernoulli}(g_T(x)).$$

Analyse des termes
Partie linéaire: x1+0.8x^2 corresp ond à un classificateur linéaire.

Partie sinusoïdale: 0.5sin(2x1) ajoute : une non-linéarité,une frontière courbe.
Pourquoi ce choix ?

Si tout était linéaire : adaptation trop facile, problème peu intéres sant. Avec la sinusoïde :
la frontière devient complexe,
le modèle doit mieux généraliser

On a donc généralement $D_S \neq D_T$ et $f_S \neq f_T$.

[4]:
```python
def make_rotation_matrix(angle: float) -> np.ndarray:
    return np.array([
        [np.cos(angle), -np.sin(angle)],
        [np.sin(angle), np.cos(angle)],
    ])


def transform_source_to_target(
    X: np.ndarray,
    angle: float = 0.45,
    translation: Tuple[float, float] = (0.8, -0.4),
    scale: Tuple[float, float] = (1.2, 0.8),
) -> np.ndarray:
    return X @ np.diag(scale) @ make_rotation_matrix(angle).T + np.
 ↪array(translation)


def target_score(X: np.ndarray) -> np.ndarray:
    x1, x2 = X[:, 0], X[:, 1]
    return 0.9 * x1 + x2 + 0.5 * np.sin(2.0 * x1 + 0.4)


def target_label_probability(X: np.ndarray, noise_level: float) -> np.ndarray:
    return sigmoid(target_score(X) / noise_level)


def target_label_function(
    X: np.ndarray,
    noise_level: float,
    rng: np.random.Generator,
) -> np.ndarray:
    return rng.binomial(n=1, p=target_label_probability(X, noise_level))
```

### 4.4 I.4 — Génération complète des échantillons source et target

On génère $n_S = 1000$ observations source et $n_T = 300$ observations target. Cette asymétrie représente le cas classique où les données source sont plus nombreuses que les données target.

[5]:
```python
def generate_domains(
    n_source: int = 1000,
    n_target: int = 300,
    noise_level: float = 0.7,
    random_state: int = 42,
) -> Tuple[DomainData, DomainData]:
    rng = np.random.default_rng(random_state)
```

random_state fixé a 42 pour la reproductibilité.
À chaque exécution : mêmes données,
mêmes figures, mêmes résultats expérimentaux.

```
    X_source = generate_source_features(n_source, rng)
    y_source = source_label_function(X_source, noise_level, rng)

    X_target_base = generate_source_features(n_target, rng)
    X_target = transform_source_to_target(X_target_base)
    y_target = target_label_function(X_target, noise_level, rng)

    return (
        DomainData(X_source, y_source, "source"),
        DomainData(X_target, y_target, "target"),
    )


source, target = generate_domains()

for domain in [source, target]:
    print(f"Domaine {domain.name}")
    print("X shape:", domain.X.shape)
    print("y shape:", domain.y.shape)
    print("Proportion de classe 1:", round(domain.y.mean(), 3))
    print()
```

génère XSDS: les donnees d'entrées

genère YSXS=xBernoulli(gS(x)).Do

```
Domaine source
X shape: (1000, 2)
y shape: (1000,)
Proportion de classe 1: 0.512

Domaine target
X shape: (300, 2)
y shape: (300,)
Proportion de classe 1: 0.553
```

Interprétation
X shape: (1000,2) : 1000 observations : x=(x1,x2).
y shape: (1000,) : 1000 labels binaires.
Proportion de classe 1: 0.512
Environ : 51.2% des exemples appartiennent à la classe positive.

### 4.5 I.5 — Visualisation des distributions et des labels

La couleur indique le label réel. Pour le domaine source, les labels viennent de $f_S$ ; pour le domaine target, ils viennent de $f_T$.

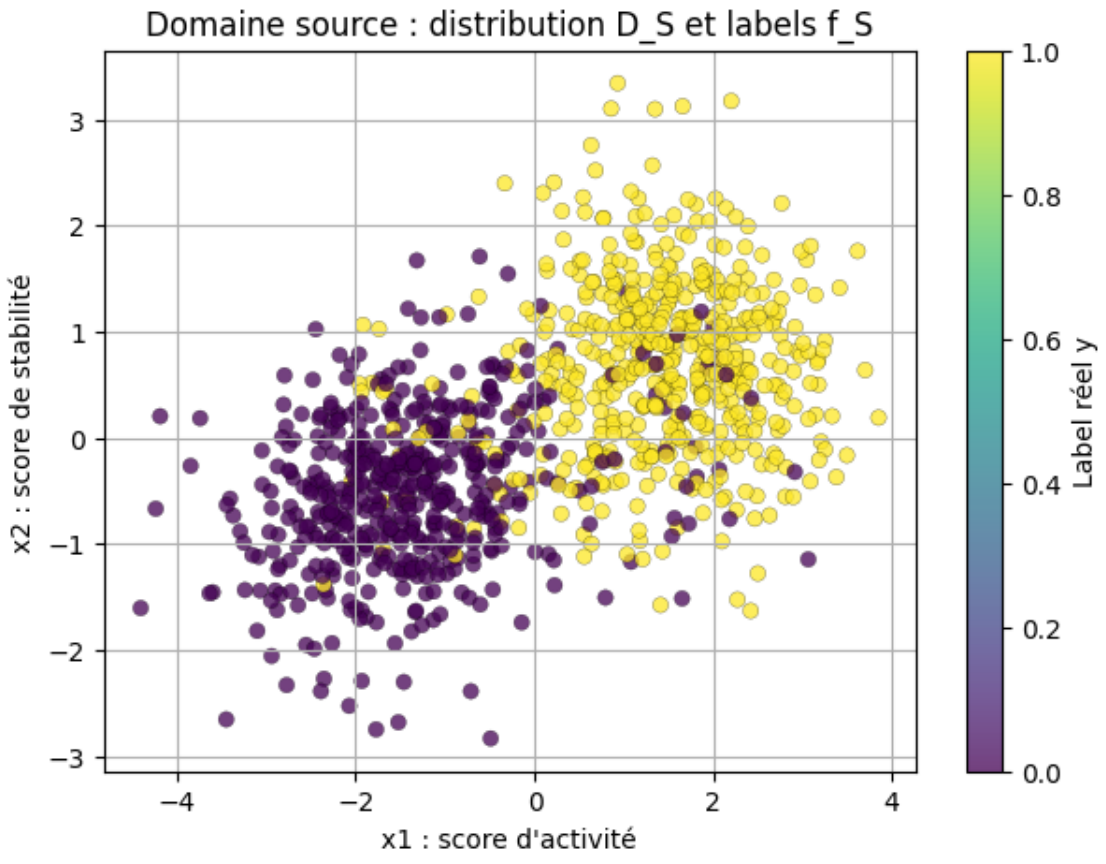
[6]:
```
def plot_single_domain(domain: DomainData, title: str) -> None:
    plt.figure()
    scatter = plt.scatter(
        domain.X[:, 0], domain.X[:, 1],
        c=domain.y, alpha=0.75, edgecolor="k", linewidth=0.2,
    )
    plt.xlabel("x1 : score d'activité")
    plt.ylabel("x2 : score de stabilité")
    plt.title(title)
    plt.colorbar(scatter, label="Label réel y")
```

cette partie permet de voir :
le décalage entre distributions,
les différences de labels,
la difficulté du transfert.

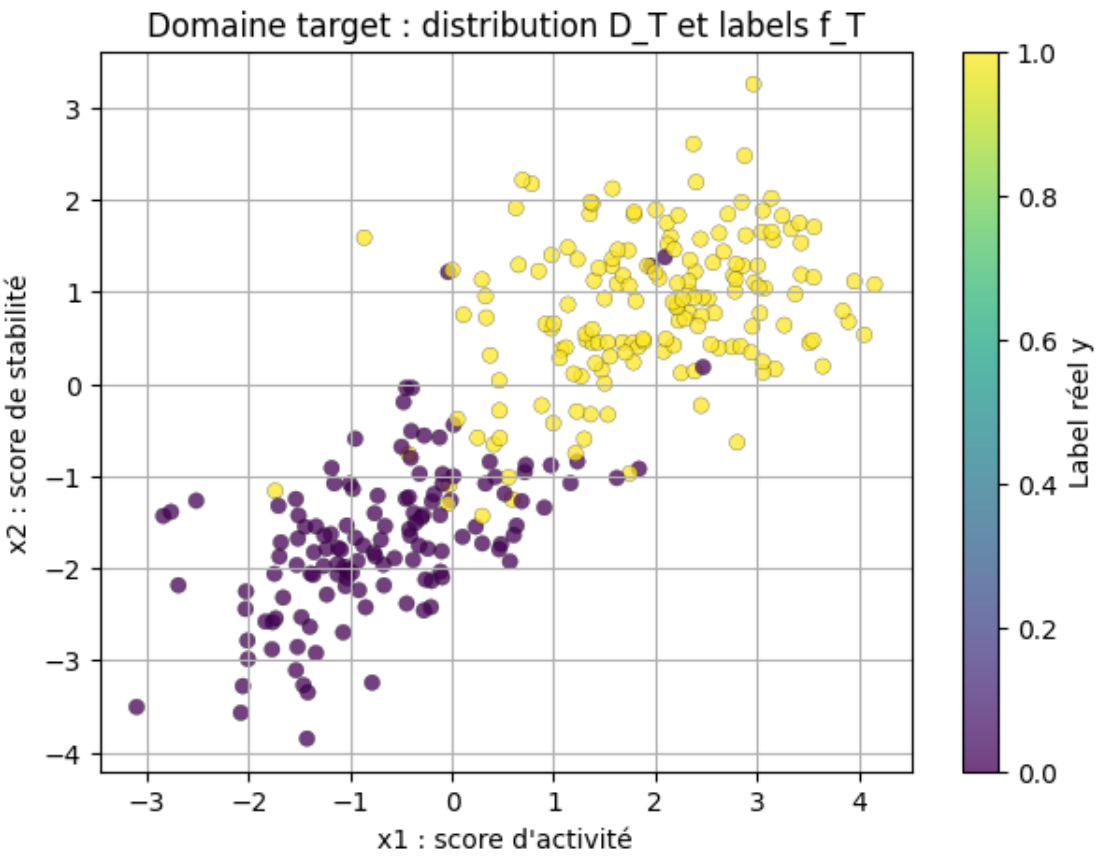
```
    plt.show()


def plot_domains_together(source: DomainData, target: DomainData) -> None:
    plt.figure(figsize=(8, 6))
    plt.scatter(source.X[:, 0], source.X[:, 1], c=source.y, marker="o", alpha=0.
 ↪55, label="Source", edgecolor="k", linewidth=0.2)
    plt.scatter(target.X[:, 0], target.X[:, 1], c=target.y, marker="x", alpha=0.
 ↪85, label="Target")
    plt.xlabel("x1 : score d'activité")
    plt.ylabel("x2 : score de stabilité")
    plt.title("Comparaison des domaines source et target")
    plt.legend()
    plt.show()


plot_single_domain(source, "Domaine source : distribution D_S et labels f_S")
plot_single_domain(target, "Domaine target : distribution D_T et labels f_T")
plot_domains_together(source, target)
```

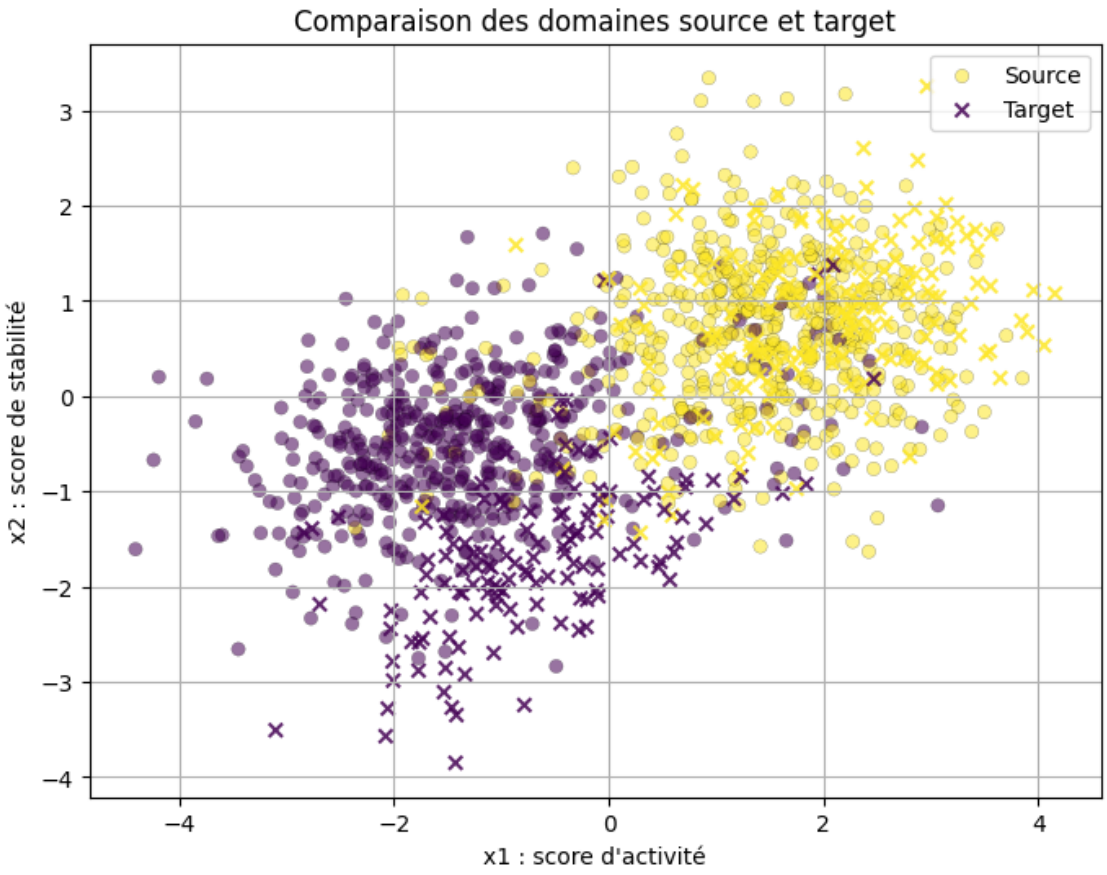



Les couleurs correspondent aux labels observés :YS ou YT




Ce qu’on doit observer
1. Décalage géométrique
Les nuages : ne sont pas superposés, sont décalés, tournés. donc Ds!=Dt

Donc :
Les graphiques montrent clairement le problème d’adaptation de domaine : les données source et cible ne suivent pas exactement la même distribution, mais elles restent suffisamment proches pour qu’un transfert soit envisageable. L’objectif sera donc d’exploiter les nombreuses données source afin d’améliorer les performances sur le domaine cible malgré ce décalage. »




### 4.6 I.6 — Visualisation des probabilités d'étiquetage vraies

Comme les données sont synthétiques, les probabilités vraies sont connues :

$$g_S(x) = P_S(Y = 1 \mid X = x), \qquad g_T(x) = P_T(Y = 1 \mid X = x).$$

Ces surfaces représentent les probabilités $g_S$ et $g_T$, et non les labels déterministes $f_S$ et $f_T$.

La ligne de niveau 0.5 correspond à la frontière de décision déterministe :

$$f(x) = \mathbf{1}\left[g(x) \geq \frac{1}{2}\right].$$

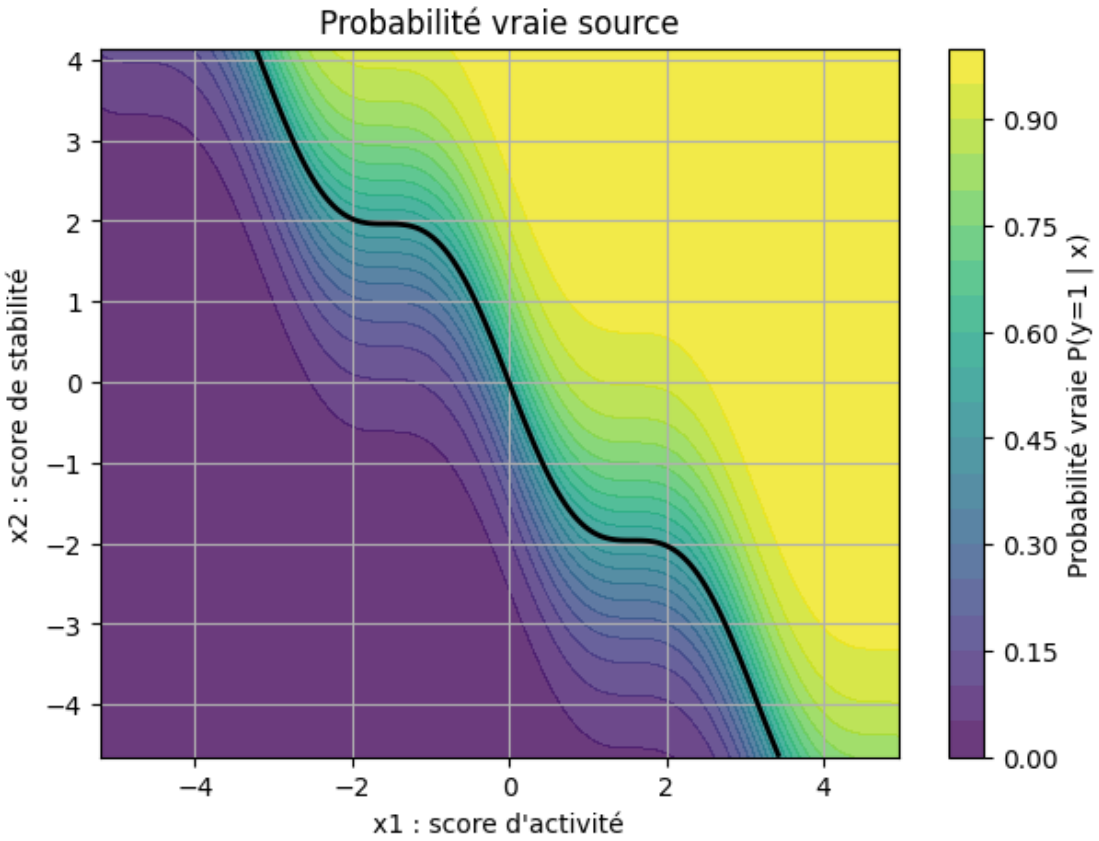
[7]:
```python
def plot_label_probability_surface(
    probability_function: Callable[[np.ndarray, float], np.ndarray],
    title: str,
    noise_level: float = 0.7,
) -> None:
    x_min = min(source.X[:, 0].min(), target.X[:, 0].min()) - 0.8
```

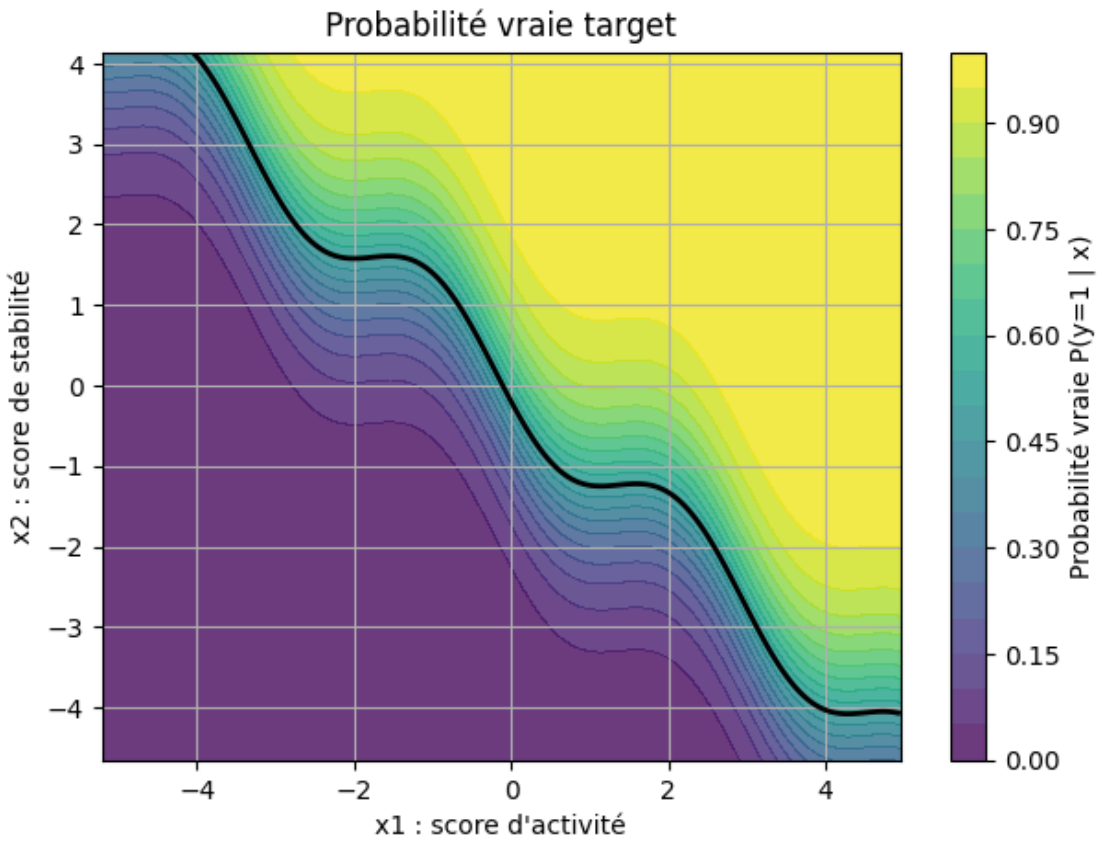
```
    x_max = max(source.X[:, 0].max(), target.X[:, 0].max()) + 0.8
    y_min = min(source.X[:, 1].min(), target.X[:, 1].min()) - 0.8
    y_max = max(source.X[:, 1].max(), target.X[:, 1].max()) + 0.8

    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min,␣
 ↪y_max, 200))
    grid = np.column_stack([xx.ravel(), yy.ravel()])
    zz = probability_function(grid, noise_level).reshape(xx.shape)

    plt.figure(figsize=(7, 5))
    surface = plt.contourf(xx, yy, zz, levels=20, alpha=0.8)
    plt.contour(xx, yy, zz, levels=[0.5], colors="black", linewidths=2)
    plt.xlabel("x1 : score d'activité")
    plt.ylabel("x2 : score de stabilité")
    plt.title(title)
    plt.colorbar(surface, label="Probabilité vraie P(y=1 | x)")
    plt.show()


plot_label_probability_surface(source_label_probability, "Probabilité vraie␣
 ↪source")
plot_label_probability_surface(target_label_probability, "Probabilité vraie␣
 ↪target")
```


Probabilité vraie source
x2 : score de stabilité
x1 : score d'activité
Probabilité vraie P(y=1 | x)
4
3
2
1
0
−1
−2
−3
−4
−4
−2
0
2
4
0.90
0.75
0.60
0.45
0.30
0.15
0.00




### 4.7 I.7 — Bilan de la génération des données

Nous avons construit deux domaines synthétiques :

$$\mathcal{D}_S = (D_S, f_S), \qquad \mathcal{D}_T = (D_T, f_T),$$

avec :

$$D_S \neq D_T \qquad \text{et} \qquad f_S \neq f_T.$$

Le domaine target est obtenu par rotation, changement d'échelle et translation de la distribution source. Sa fonction d'étiquetage est également légèrement modifiée. Les deux domaines gardent néanmoins une structure commune : le transfert est donc possible, mais non trivial.

Dans la partie suivante, on apprend une hypothèse $h_S$ sur le domaine source, puis on compare :

$$\hat{\epsilon}_S(h_S) \quad \text{et} \quad \hat{\epsilon}_T(h_S).$$

# 5 Partie II — Estimation du modèle source et première borne d'adaptation

Nous avons maintenant un domaine source :

$$\mathcal{D}_S = (D_S, f_S).$$

Le but de cette partie est d'apprendre, à partir d'un échantillon source labellisé, une hypothèse (h_S) qui approxime la fonction d'étiquetage vraie $f_S$.

Autrement dit, on cherche une fonction :

$$h_S : \mathcal{X} \to \{0, 1\}$$

telle que :

$$h_S(x) \approx f_S(x).$$

Dans notre exemple, (x=(x_1,x_2)) représente un profil client fictif décrit par deux scores numériques. Le label vaut :

$$y = 1 \quad \text{si le profil est à risque élevé,}$$

et

$$y = 0 \quad \text{si le profil est à risque faible.}$$

## 5.1 II.1 — Définition : classe d'hypothèses $\mathcal{H}$ et représentation $\varphi$

La vraie fonction d'étiquetage source a été construite à partir du score :

$$g_S(x) = x_1 + 0.8x_2 + 0.5\sin(2x_1).$$

Puis le label est tiré selon :

$$\mathbb{P}(Y = 1 \mid X = x) = \sigma\left(\frac{g_S(x)}{\eta}\right),$$

où $\eta$ désigne le niveau de bruit et $\sigma$ la fonction sigmoïde.

Pour apprendre une hypothèse raisonnable sans rendre le modèle trop complexe, on utilise une régression logistique sur la transformation :

$$\varphi(x) = (x_1, x_2, \sin(2x_1)).$$

La classe d'hypothèses considérée est donc :

$$\mathcal{H} = \left\{ h_w(x) = \mathbf{1}\left[ w^\top \varphi(x) + b \geq 0 \right] : w \in \mathbb{R}^3, b \in \mathbb{R} \right\}.$$

Ce choix est important : le modèle reste simple et interprétable, mais il contient une variable non linéaire cohérente avec la vraie règle source. On ne donne pas au modèle la fonction exacte complète, mais on lui fournit une représentation adaptée.

[8]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, StandardScaler


def feature_map(X: np.ndarray) -> np.ndarray:
    """
    Transforme les variables originales x = (x1, x2) en variables utilisées par␣
 ↪le modèle.

    La transformation choisie est :
        phi(x) = (x1, x2, sin(2*x1))

    Cette transformation reste simple, mais elle permet au modèle linéaire de␣
 ↪capturer
    une partie de la non-linéarité présente dans la fonction d'étiquetage␣
 ↪source.
    """

    x1 = X[:, 0]
    x2 = X[:, 1]

    return np.column_stack([x1, x2, np.sin(2.0 * x1)])


def classification_error(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """
    Calcule l'erreur de classification empirique.

    Cette quantité correspond à :
        nombre de prédictions incorrectes / nombre total d'exemples
    """

    return 1.0 - accuracy_score(y_true, y_pred)
```

## 5.2 II.2 — Découpage de l'échantillon source

On dispose d'un échantillon source :

$$S_S = \{(x_i^S, y_i^S)\}_{i=1}^{m_S}.$$

Pour évaluer proprement notre hypothèse, on sépare l'échantillon source en deux parties :

$$S_S^{\text{train}} \quad \text{et} \quad S_S^{\text{test}}.$$

Le modèle est entraîné uniquement sur $S_S^{\text{train}}$, puis son erreur source est estimée sur $S_S^{\text{test}}$. Cela évite de confondre l'erreur d'entraînement et l'erreur de généralisation empirique.

[9]:
```
X_source_train, X_source_test, y_source_train, y_source_test = train_test_split(
    source.X,
    source.y,
    test_size=0.30,
    random_state=42,
    stratify=source.y,
)

print("Taille de l'ensemble source d'entraînement :", X_source_train.shape)
print("Taille de l'ensemble source de test         :", X_source_test.shape)
print("Proportion de classe 1 dans train          :", y_source_train.mean().
 ↪round(3))
print("Proportion de classe 1 dans test           :", y_source_test.mean().
 ↪round(3))
```

```
Taille de l'ensemble source d'entraînement : (700, 2)
Taille de l'ensemble source de test         : (300, 2)
Proportion de classe 1 dans train          : 0.511
Proportion de classe 1 dans test           : 0.513
```

### 5.3 II.3 — Estimation du modèle $h_S$ par minimisation du risque empirique

On entraîne une régression logistique. Elle estime une probabilité conditionnelle :

$$\hat{p}_S(y = 1 \mid x) = \sigma\left(w^\top \varphi(x) + b\right).$$

La règle de décision associée est :

$$h_S(x) = \mathbf{1}\left[\hat{p}_S(y = 1 \mid x) \geq \frac{1}{2}\right].$$

De manière équivalente :

$$h_S(x) = \mathbf{1}\left[w^\top \varphi(x) + b \geq 0\right].$$

L'apprentissage consiste à choisir les paramètres $(w, b)$ qui réduisent l'erreur de prédiction sur les données source d'entraînement.

[10]:
```python
source_model = Pipeline(
    steps=[
        ("features", FunctionTransformer(feature_map, validate=False)),
        ("scaler", StandardScaler()),
        ("classifier", LogisticRegression(max_iter=1000, random_state=42)),
    ]
)

source_model.fit(X_source_train, y_source_train)

print("Modèle h_S entraîné sur le domaine source.")
```

```
Modèle h_S entraîné sur le domaine source.
```

### 5.4 II.4 — Paramètres appris par le modèle

La régression logistique apprend un vecteur de poids associé aux variables :

$$\varphi_1(x) = x_1, \quad \varphi_2(x) = x_2, \quad \varphi_3(x) = \sin(2x_1).$$

Les coefficients ne doivent pas être interprétés comme les coefficients exacts de $f_S$, car le modèle utilise une standardisation avant la régression logistique. Ils permettent néanmoins de vérifier quelles variables sont utilisées par le modèle.

[11]:
```python
classifier = source_model.named_steps["classifier"]

feature_names = ["x1", "x2", "sin(2*x1)"]
coefficients = classifier.coef_[0]
intercept = classifier.intercept_[0]

for name, coef in zip(feature_names, coefficients):
    print(f"Coefficient associé à {name:10s} : {coef: .4f}")

print(f"Intercept : {intercept: .4f}")
```

```
Coefficient associé à x1         :  2.4611
Coefficient associé à x2         :  0.9896
Coefficient associé à sin(2*x1)  :  0.6178
Intercept :  0.1066
```

### 5.5 II.5 — Calcul des erreurs empiriques source et target

L'erreur empirique source de l'hypothèse apprise est :

$$\hat{\epsilon}_S(h_S) = \frac{1}{m_S} \sum_{i=1}^{m_S} \mathbf{1}\left[h_S(x_i^S) \neq y_i^S\right].$$

On calcule cette erreur sur l'ensemble d'entraînement et sur l'ensemble de test source. L'erreur de test source est celle qui nous intéresse le plus à ce stade, car elle donne une première indication de la capacité de $h_S$ à approximer $f_S$ sur des données source non vues pendant l'apprentissage.

[12]:
```
y_source_train_pred = source_model.predict(X_source_train)
y_source_test_pred = source_model.predict(X_source_test)

source_train_error = classification_error(y_source_train, y_source_train_pred)
source_test_error = classification_error(y_source_test, y_source_test_pred)

print(f"Erreur source sur train  : {source_train_error:.4f}")
print(f"Erreur source sur test   : {source_test_error:.4f}")
```

```
Erreur source sur train  : 0.0871
Erreur source sur test   : 0.0933
```

### 5.6 II.6 — Visualisation de la frontière de décision apprise

Nous comparons maintenant deux objets :

1. la frontière vraie associée à la fonction d'étiquetage source $f_S$, donnée par :

$$g_S(x) = 0,$$

2. la frontière apprise par le modèle $h_S$, donnée par :

$$w^\top \varphi(x) + b = 0.$$

Si le modèle approxime correctement $f_S$, ces deux frontières doivent être proches sur la zone où les données source sont concentrées.

[13]:
```
def plot_source_decision_boundary(model: Pipeline, source_domain: DomainData)␣
↪-> None:
    """
    Affiche les données source, la frontière vraie de f_S et la frontière␣
↪apprise par h_S.
    """

    x_min, x_max = source_domain.X[:, 0].min() - 1.0, source_domain.X[:, 0].
↪max() + 1.0
    y_min, y_max = source_domain.X[:, 1].min() - 1.0, source_domain.X[:, 1].
↪max() + 1.0

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 300),
        np.linspace(y_min, y_max, 300),
    )
```

```
    grid = np.column_stack([xx.ravel(), yy.ravel()])

    # Probabilité prédite par h_S
    predicted_proba = model.predict_proba(grid)[:, 1].reshape(xx.shape)

    # Probabilité vraie associée à la fonction source f_S
    true_proba = source_label_probability(grid, noise_level=0.7).reshape(xx.
 ↪shape)

    plt.figure(figsize=(8, 6))

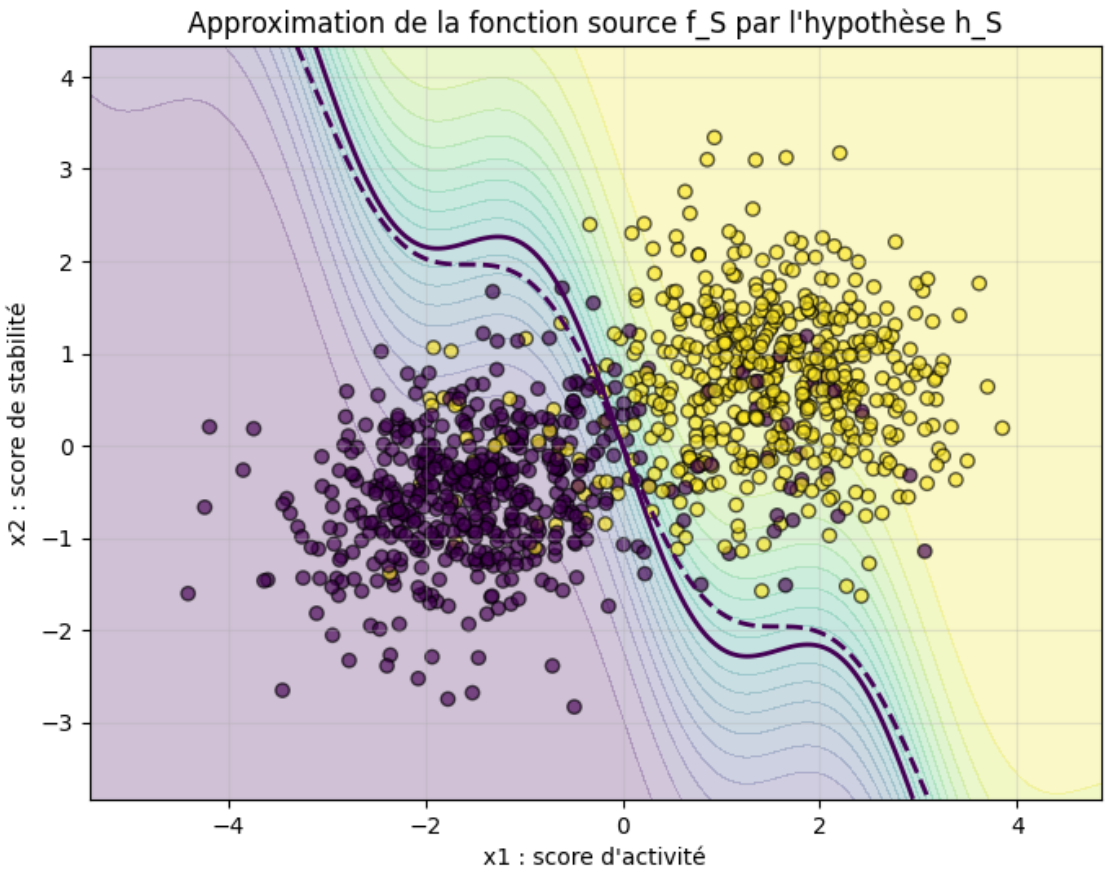
    plt.contourf(xx, yy, predicted_proba, levels=20, alpha=0.25)
    plt.scatter(source_domain.X[:, 0], source_domain.X[:, 1], c=source_domain.
 ↪y, alpha=0.65, edgecolor="k")

    # Frontière apprise : p_h(y=1|x) = 0.5
    plt.contour(xx, yy, predicted_proba, levels=[0.5], linewidths=2)

    # Frontière vraie : p_S(y=1|x) = 0.5
    plt.contour(xx, yy, true_proba, levels=[0.5], linestyles="--", linewidths=2)

    plt.xlabel("x1 : score d'activité")
    plt.ylabel("x2 : score de stabilité")
    plt.title("Approximation de la fonction source f_S par l'hypothèse h_S")
    plt.grid(alpha=0.3)
    plt.show()


plot_source_decision_boundary(source_model, source)
```




### 5.7 II.7 — Interprétation expérimentale de $h_S$

À ce stade, nous avons construit une première hypothèse :

$$h_S \in \mathcal{H}.$$

Cette hypothèse a été apprise uniquement à partir des données du domaine source. Elle cherche donc à approximer la vraie fonction d'étiquetage source :

$$f_S : \mathcal{X} \to \{0, 1\}.$$

Le point important est que $h_S$ n'est pas encore un modèle adapté au domaine target. C'est seulement un modèle source.

La prochaine étape consistera donc à appliquer cette même hypothèse $h_S$ au domaine target afin de comparer :

$$\hat{\epsilon}_S(h_S) \quad \text{et} \quad \hat{\epsilon}_T(h_S).$$

Cette comparaison permettra de mesurer l'effet du changement de domaine sur l'erreur du modèle.

## 5.8 II.8 — Théorème 1 : première borne avec $d_1(D_S, D_T)$

Nous avons construit deux domaines :

$$\mathcal{D}_S = (D_S, f_S)$$

et

$$\mathcal{D}_T = (D_T, f_T).$$

Le modèle appris précédemment est une hypothèse $h_S$, entraînée uniquement sur le domaine source afin d'approcher la fonction d'étiquetage source $f_S$.

Le théorème 1 donne la borne suivante, pour toute hypothèse $h$ :

$$\epsilon_T(h) \leq \epsilon_S(h) + d_1(D_S, D_T) + \min\left\{\mathbb{E}_{x\sim D_S}|f_S(x) - f_T(x)|, \mathbb{E}_{x\sim D_T}|f_S(x) - f_T(x)|\right\}.$$

Le premier terme $\epsilon_S(h)$ est l'erreur de l'hypothèse sur le domaine source.

Le deuxième terme $d_1(D_S, D_T)$ mesure l'écart entre les distributions source et target :

$$d_1(D_S, D_T) = 2\sup_B \left|\mathbb{P}_{D_S}(B) - \mathbb{P}_{D_T}(B)\right|.$$

Ce terme est théoriquement important, mais il est difficile à calculer directement car le supremum porte sur tous les ensembles mesurables $B$.

Le troisième terme mesure la différence entre les vraies fonctions d'étiquetage des deux domaines. Comme les données sont synthétiques, on connaît directement $f_S(x)$ et $f_T(x)$ pour chaque point $x$. On peut donc construire un tableau point par point, puis prendre la moyenne des écarts.

### 5.8.1 II.8.0 — Demontsration Simplifiée du Theoreme 1

L'idée de la preuve est classique : on part de l'erreur cible

$$\varepsilon_T(h)$$

et on ajoute puis retranche des quantités intermédiaires afin de séparer les différentes sources de l'erreur.

On rappelle que :

$$\varepsilon_T(h) = \mathbb{E}_{x\sim D_T}\left[|h(x) - f_T(x)|\right].$$

L'objectif est de contrôler cette erreur cible à partir de trois contributions :

1. l'erreur du modèle sur le domaine source ;

2. le changement de fonction d'étiquetage entre la source et la cible ;
3. le changement de distribution entre la source et la cible.

---

**1. Ajout et soustraction de termes intermédiaires** On part de :

$$\varepsilon_T(h).$$

On ajoute et on retranche d'abord l'erreur source associée à la fonction cible $f_T$, puis l'erreur source associée à la fonction source $f_S$ :

$$\varepsilon_T(h) = \varepsilon_T(h) + \varepsilon_S(h, f_T) - \varepsilon_S(h, f_T) + \varepsilon_S(h, f_S) - \varepsilon_S(h, f_S).$$

Comme $\varepsilon_S(h, f_S) = \varepsilon_S(h)$, on obtient :

$$\varepsilon_T(h) = \varepsilon_S(h) + (\varepsilon_S(h, f_T) - \varepsilon_S(h, f_S)) + (\varepsilon_T(h) - \varepsilon_S(h, f_T)) \,.$$

Cette égalité permet de faire apparaître trois termes distincts :

$$\varepsilon_T(h) = \underbrace{\varepsilon_S(h)}_{\text{erreur source}} + \underbrace{(\varepsilon_S(h, f_T) - \varepsilon_S(h, f_S))}_{\text{changement de labels}} + \underbrace{(\varepsilon_T(h) - \varepsilon_S(h, f_T))}_{\text{changement de distribution}} .$$

---

**2. Passage à une borne supérieure** Les deux derniers termes peuvent être positifs ou négatifs. Pour obtenir une borne supérieure, on utilise la valeur absolue :

$$\varepsilon_T(h) \leq \varepsilon_S(h) + |\varepsilon_S(h, f_T) - \varepsilon_S(h, f_S)| + |\varepsilon_T(h) - \varepsilon_S(h, f_T)| \,.$$

Cette étape repose sur l'inégalité triangulaire :

$$a + b + c \leq a + |b| + |c|.$$

Ainsi, on a séparé l'erreur cible en trois contributions positives.

---

**3. Interprétation des trois termes** Le premier terme est :

$$\varepsilon_S(h).$$

Il correspond à l'erreur du modèle $h$ sur le domaine source.
C'est l'erreur que l'on peut estimer directement à partir des données sources étiquetées.

---

Le deuxième terme est :

$$|\varepsilon_S(h, f_T) - \varepsilon_S(h, f_S)|\,.$$

Il mesure le changement de labels entre la fonction source $f_S$ et la fonction cible $f_T$, tout en restant sur la distribution source $D_S$.

Autrement dit, ce terme quantifie l'écart dû au fait que la règle d'étiquetage peut changer entre le domaine source et le domaine cible.

---

Le troisième terme est :

$$|\varepsilon_T(h) - \varepsilon_S(h, f_T)|\,.$$

Il mesure le changement de distribution entre $D_S$ et $D_T$, en gardant la même fonction d'étiquetage cible $f_T$.

En effet, on compare ici deux erreurs calculées avec la même fonction $f_T$, mais sous deux distributions différentes.

---

**4. Contrôle du terme de distribution** On écrit les deux erreurs sous forme intégrale :

$$\varepsilon_T(h, f_T) = \int \phi_T(x)|h(x) - f_T(x)|dx,$$

et

$$\varepsilon_S(h, f_T) = \int \phi_S(x)|h(x) - f_T(x)|dx.$$

Donc :

$$|\varepsilon_T(h, f_T) - \varepsilon_S(h, f_T)| = \left|\int (\phi_T(x) - \phi_S(x))\,|h(x) - f_T(x)|dx\right|.$$

En utilisant l'inégalité triangulaire pour les intégrales, on obtient :

$$\left|\int (\phi_T(x) - \phi_S(x))\,|h(x) - f_T(x)|dx\right| \leq \int |\phi_T(x) - \phi_S(x)|\,|h(x) - f_T(x)|dx.$$

Comme $|h(x) - f_T(x)| \leq 1$, on a :

$$\int |\phi_T(x) - \phi_S(x)|\,|h(x) - f_T(x)|dx \leq \int |\phi_T(x) - \phi_S(x)|\,dx.$$

Or,

$$\int |\phi_T(x) - \phi_S(x)|\, dx = d_1(D_S, D_T).$$

Ainsi :

$$|\varepsilon_T(h, f_T) - \varepsilon_S(h, f_T)| \leq d_1(D_S, D_T).$$

---

**5. Conclusion** En regroupant les termes précédents, on obtient la borne :

$$\varepsilon_T(h) \leq \varepsilon_S(h) + |\varepsilon_S(h, f_T) - \varepsilon_S(h, f_S)| + d_1(D_S, D_T).$$

Cette inégalité montre que l'erreur cible est contrôlée par trois quantités :

$$\boxed{\varepsilon_T(h) \leq \varepsilon_S(h) + \text{écart de labels} + \text{écart de distribution}}$$

Ainsi, pour bien généraliser au domaine cible, il faut simultanément :

- apprendre un modèle performant sur la source ;
- limiter le changement de règle d'étiquetage ;
- contrôler l'écart entre les distributions source et cible.

[14]:
```python
def true_source_label_probability(X: np.ndarray, noise_level: float = 0.7) ->␣
 ↪np.ndarray:
    """Probabilité vraie g_S(x)=P_S(Y=1|X=x)."""
    return source_label_probability(X, noise_level)


def true_target_label_probability(X: np.ndarray, noise_level: float = 0.7) ->␣
 ↪np.ndarray:
    """Probabilité vraie g_T(x)=P_T(Y=1|X=x)."""
    return target_label_probability(X, noise_level)


def true_source_label(
    X: np.ndarray,
    noise_level: float = 0.7,
    threshold: float = 0.5,
) -> np.ndarray:
    """Fonction d'étiquetage déterministe f_S(x) dans {0,1}."""
    return (true_source_label_probability(X, noise_level) >= threshold).
 ↪astype(int)


def true_target_label(
    X: np.ndarray,
```

```
    noise_level: float = 0.7,
    threshold: float = 0.5,
) -> np.ndarray:
    """Fonction d'étiquetage déterministe f_T(x) dans {0,1}."""
    return (true_target_label_probability(X, noise_level) >= threshold).
 ↪astype(int)
```

### 5.8.2 II.8.1 — Estimation point par point de l'écart entre $f_S$ et $f_T$

On distingue bien :

$$g_S(x), g_T(x) \in [0,1],$$

qui sont des probabilités d'étiquetage, et

$$f_S(x), f_T(x) \in \{0,1\},$$

qui sont les fonctions d'étiquetage déterministes.

Pour chaque point (x_i), on calcule donc :

$$g_S(x_i), \qquad g_T(x_i), \qquad f_S(x_i), \qquad f_T(x_i), \qquad |f_S(x_i) - f_T(x_i)|.$$

Les deux quantités du théorème sont ensuite estimées par moyenne :

$$\widehat{\Delta}_S = \frac{1}{n_S} \sum_{i=1}^{n_S} |f_S(x_i^S) - f_T(x_i^S)|,$$

et

$$\widehat{\Delta}_T = \frac{1}{n_T} \sum_{i=1}^{n_T} |f_S(x_i^T) - f_T(x_i^T)|.$$

Le terme utilisé dans le théorème est alors :

$$\widehat{\Delta}_f = \min(\widehat{\Delta}_S, \widehat{\Delta}_T).$$

[15]:
```
def build_label_gap_table(
    X: np.ndarray,
    domain_name: str,
    noise_level: float = 0.7,
) -> pd.DataFrame:
    g_s = true_source_label_probability(X, noise_level)
    g_t = true_target_label_probability(X, noise_level)
```

```
    f_s = true_source_label(X, noise_level)
    f_t = true_target_label(X, noise_level)

    return pd.DataFrame({
        "domaine": domain_name,
        "x1": X[:, 0],
        "x2": X[:, 1],
        "g_S(x)=P_S(Y=1|x)": g_s,
        "g_T(x)=P_T(Y=1|x)": g_t,
        "f_S(x)": f_s,
        "f_T(x)": f_t,
        "|f_S(x)-f_T(x)|": np.abs(f_s - f_t),
    })


label_gap_source = build_label_gap_table(source.X, "source")
label_gap_target = build_label_gap_table(target.X, "target")

Delta_source = label_gap_source["|f_S(x)-f_T(x)|"].mean()
Delta_target = label_gap_target["|f_S(x)-f_T(x)|"].mean()

labeling_difference = {
    "E_DS_abs_fS_minus_fT": Delta_source,
    "E_DT_abs_fS_minus_fT": Delta_target,
    "min_labeling_difference": min(Delta_source, Delta_target),
}

print("Vérification : f_S et f_T sont bien binaires")
print("-" * 50)
print("Valeurs de f_S :", [int(v) for v in sorted(label_gap_source["f_S(x)"].
 ↪unique())])
print("Valeurs de f_T :", [int(v) for v in sorted(label_gap_source["f_T(x)"].
 ↪unique())])
print()

print("Tableau source : premières lignes")
display(label_gap_source.head(8).round(4))

print("Tableau target : premières lignes")
display(label_gap_target.head(8).round(4))

print("Moyennes des écarts")
print("-" * 40)
print(f"Delta_S = {Delta_source:.4f}")
print(f"Delta_T = {Delta_target:.4f}")
print(f"min(Delta_S, Delta_T) = {labeling_difference['min_labeling_difference']:
 ↪.4f}")
```

```
Vérification : f_S et f_T sont bien binaires
------------------------------------------------
Valeurs de f_S : [0, 1]
Valeurs de f_T : [0, 1]

Tableau source : premières lignes
```

| | domaine | x1 | x2 | … | f_S(x) | f_T(x) | \|f_S(x)-f_T(x)\| |
|---|---|---|---|---|---|---|---|
| 0 | source | 0.8817 | 0.1120 | … | 1 | 1 | 0 |
| 1 | source | 2.0971 | -0.0604 | … | 1 | 1 | 0 |
| 2 | source | 0.9102 | -0.4232 | … | 1 | 1 | 0 |
| 3 | source | -0.7111 | -2.3903 | … | 0 | 0 | 0 |
| 4 | source | 0.8709 | 1.0185 | … | 1 | 1 | 0 |
| 5 | source | -2.2024 | 0.2288 | … | 0 | 0 | 0 |
| 6 | source | -0.7426 | 1.1657 | … | 0 | 1 | 1 |
| 7 | source | 2.6906 | 1.1230 | … | 1 | 1 | 0 |

```
[8 rows x 8 columns]

Tableau target : premières lignes
```

| | domaine | x1 | x2 | … | f_S(x) | f_T(x) | \|f_S(x)-f_T(x)\| |
|---|---|---|---|---|---|---|---|
| 0 | target | -1.0705 | -2.6967 | … | 0 | 0 | 0 |
| 1 | target | -1.3980 | -2.6363 | … | 0 | 0 | 0 |
| 2 | target | -2.8400 | -1.4332 | … | 0 | 0 | 0 |
| 3 | target | 0.5917 | -1.2531 | … | 1 | 0 | 1 |
| 4 | target | -1.7393 | -1.1608 | … | 0 | 0 | 0 |
| 5 | target | -1.3907 | -2.0582 | … | 0 | 0 | 0 |
| 6 | target | 1.6235 | 1.0855 | … | 1 | 1 | 0 |
| 7 | target | 1.6758 | 0.4512 | … | 1 | 1 | 0 |

```
[8 rows x 8 columns]

Moyennes des écarts
---------------------------------------
Delta_S = 0.0160
Delta_T = 0.0200
min(Delta_S, Delta_T) = 0.0160
```

#### 5.8.3 II.8.2 — Comparaison point par point entre $f$ et $h_S$

Nous avons entraîné $h_S$ sur le domaine source. Pour chaque point, on compare maintenant :

- le label observé $y_i$, généré à partir de la vraie fonction d'étiquetage ;
- la prédiction $h_S(x_i)$ ;
- l'indicateur d'erreur $\mathbf{1}_{\{h_S(x_i) \neq y_i\}}$.

Les erreurs empiriques sont ensuite les moyennes de cette dernière colonne :

$$\hat{\epsilon}_S(h_S) = \frac{1}{n_S} \sum_{i=1}^{n_S} \mathbf{1}_{\{h_S(x_i^S) \neq y_i^S\}},$$

et

$$\hat{\epsilon}_T(h_S) = \frac{1}{n_T} \sum_{i=1}^{n_T} \mathbf{1}_{\{h_S(x_i^T) \neq y_i^T\}}.$$

[16]:
```python
def build_error_table(domain: DomainData, model: Pipeline) -> pd.DataFrame:
    h_pred = model.predict(domain.X)
    h_proba = model.predict_proba(domain.X)[:, 1]

    return pd.DataFrame({
        "domaine": domain.name,
        "x1": domain.X[:, 0],
        "x2": domain.X[:, 1],
        "y": domain.y,
        "h_S(x)": h_pred,
        "P_h(y=1|x)": h_proba,
        "erreur": (h_pred != domain.y).astype(int),
    })


error_source = build_error_table(source, source_model)
error_target = build_error_table(target, source_model)

epsilon_source = error_source["erreur"].mean()
epsilon_target = error_target["erreur"].mean()

print("Tableau source : premières lignes")
display(error_source.head(8).round(4))

print("Tableau target : premières lignes")
display(error_target.head(8).round(4))

print("Erreurs empiriques de h_S")
print("-" * 40)
print(f"epsilon_S(h_S) = {epsilon_source:.4f}")
print(f"epsilon_T(h_S) = {epsilon_target:.4f}")
print(f"epsilon_T(h_S) - epsilon_S(h_S) = {epsilon_target - epsilon_source:.
  ↪4f}")
```

```
Tableau source : premières lignes
```

| | domaine | x1 | x2 | y | h_S(x) | P_h(y=1\|x) | erreur |
|---|---|---|---|---|---|---|---|
| 0 | source | 0.8817 | 0.1120 | 1 | 1 | 0.9017 | 0 |
| 1 | source | 2.0971 | -0.0604 | 1 | 1 | 0.8969 | 0 |

```
2  source  0.9102 -0.4232  1       1      0.8467       0
3  source -0.7111 -2.3903  0       0      0.0141       0
4  source  0.8709  1.0185  1       1      0.9575       0
5  source -2.2024  0.2288  0       0      0.1142       0
6  source -0.7426  1.1657  0       0      0.3253       0
7  source  2.6906  1.1230  1       1      0.9861       0

Tableau target : premières lignes

  domaine      x1      x2  y  h_S(x)  P_h(y=1|x)  erreur
0  target -1.0705 -2.6967  0       0      0.0071       0
1  target -1.3980 -2.6363  0       0      0.0074       0
2  target -2.8400 -1.4332  0       0      0.0070       0
3  target  0.5917 -1.2531  1       1      0.5962       0
4  target -1.7393 -1.1608  1       0      0.0347       1
5  target -1.3907 -2.0582  0       0      0.0131       0
6  target  1.6235  1.0855  1       1      0.9645       0
7  target  1.6758  0.4512  1       1      0.9341       0

Erreurs empiriques de h_S
----------------------------------------
epsilon_S(h_S) = 0.0890
epsilon_T(h_S) = 0.0833
epsilon_T(h_S) - epsilon_S(h_S) = -0.0057
```

### 5.8.4 II.8.3 — Application partielle du Théorème 1

Le théorème 1 donne :

$$\epsilon_T(h_S) \leq \epsilon_S(h_S) + d_1(D_S, D_T) + \min\left\{\mathbb{E}_{D_S}|f_S(x) - f_T(x)|, \mathbb{E}_{D_T}|f_S(x) - f_T(x)|\right\}.$$

Dans notre expérience, nous pouvons calculer :

$$\hat{\epsilon}_S(h_S)$$

et

$$\widehat{\Delta}_f = \min(\widehat{\Delta}_S, \widehat{\Delta}_T).$$

Cependant, le terme

$$d_1(D_S, D_T) = 2 \sup_B \left|\mathbb{P}_{D_S}(B) - \mathbb{P}_{D_T}(B)\right|$$

n'est pas directement calculable en pratique, car il demande d'optimiser sur tous les ensembles mesurables (B).

On peut donc écrire une borne partielle :

$$\epsilon_T(h_S) \leq \underbrace{\hat{\epsilon}_S(h_S) + \widehat{\Delta}_f}_{\text{partie calculable}} + d_1(D_S, D_T).$$

Cette forme montre clairement pourquoi le théorème 1 est théoriquement intéressant mais difficilement exploitable numériquement.

[17]:
```
delta_f = labeling_difference["min_labeling_difference"]

partial_bound_without_d1 = epsilon_source + delta_f

print("Borne partielle issue du Théorème 1")
print("-" * 50)
print(f"epsilon_S(h_S)                          = {epsilon_source:.4f}")
print(f"min E|f_S(x) - f_T(x)|                  = {delta_f:.4f}")
print(f"Partie calculable sans d1               = {partial_bound_without_d1:.
 ↪4f}")
print()
print("Théorème 1 :")
print(
    "epsilon_T(h_S) <= "
    f"{partial_bound_without_d1:.4f} + d1(D_S, D_T)"
)
print()
print(f"Erreur target observée epsilon_T(h_S)  = {epsilon_target:.4f}")
```

```
Borne partielle issue du Théorème 1
--------------------------------------------------
epsilon_S(h_S)                          = 0.0890
min E|f_S(x) - f_T(x)|                  = 0.0160
Partie calculable sans d1               = 0.1050

Théorème 1 :
epsilon_T(h_S) <= 0.1050 + d1(D_S, D_T)

Erreur target observée epsilon_T(h_S)  = 0.0833
```

### 5.8.5 II.8.4 — Discussion : limite pratique du terme $d_1$

Nous observons que la borne du théorème 1 contient trois éléments :

1. l'erreur source de l'hypothèse $h_S$;
2. l'écart entre les deux fonctions d'étiquetage $f_S$ et $f_T$ ;
3. la divergence $d_1(D_S, D_T)$ entre les distributions.

Les deux premiers termes sont estimables dans notre cadre synthétique.

En revanche, le terme $d_1(D_S, D_T)$ est difficile à calculer, même si nous connaissons le mécanisme de génération des données. En effet, cette divergence demande de considérer tous les ensembles mesurables possibles.

C'est précisément pour résoudre cette difficulté que la suite du papier introduit une divergence plus adaptée à l'apprentissage statistique : la divergence induite par une classe d'hypothèses, notée $\mathcal{H}$-divergence puis $\mathcal{H}\Delta\mathcal{H}$-divergence.

La prochaine étape consiste donc à remplacer $d_1(D_S, D_T)$ par une quantité estimable à partir d'un classifieur de domaine.

### 5.8.6 II.8.5 Conclusion après le Théorème 1

Le Théorème 1 donne une première décomposition de l'erreur target. Il montre que l'erreur sur le domaine cible ne dépend pas uniquement de la performance du modèle sur le domaine source.

La borne obtenue fait apparaître trois contributions :

$$\varepsilon_T(h) \leq \varepsilon_S(h) + \text{écart entre les fonctions d'étiquetage} + \text{écart entre les distributions.}$$

Cette décomposition est importante car elle sépare clairement trois difficultés différentes.

La première difficulté est statistique : il faut apprendre un modèle performant sur les données sources. La deuxième difficulté vient du possible changement de règle d'étiquetage entre les domaines : même si les distributions étaient proches, le transfert peut échouer si $f_S$ et $f_T$ sont trop différentes. La troisième difficulté vient du décalage de distribution entre $D_S$ et $D_T$.

Cependant, cette première borne reste limitée en pratique. Le terme $d_1(D_S, D_T)$ est difficile à estimer directement en grande dimension, car il suppose une connaissance fine des densités source et cible. C'est précisément cette limite qui motive l'introduction de divergences plus adaptées à l'apprentissage statistique, comme la $\mathcal{H}$-divergence puis la $\mathcal{H}\Delta\mathcal{H}$-divergence.

# 6 Partie III — Du Théorème 1 au Théorème 2 : divergences et hypothèse jointe idéale

Le théorème 1 utilise la divergence

$$d_1(D_S, D_T) = 2 \sup_B \left| \mathbb{P}_{D_S}(B) - \mathbb{P}_{D_T}(B) \right|.$$

Cette divergence est très générale, mais elle est difficile à calculer car le supremum porte sur tous les ensembles mesurables $B$.

Pour obtenir une quantité plus exploitable en apprentissage statistique, on restreint les ensembles considérés à ceux qui peuvent être représentés par une classe d'hypothèses $\mathcal{H}$.

On définit alors la $\mathcal{H}$-divergence :

$$d_{\mathcal{H}}(D_S, D_T) = 2 \sup_{h \in \mathcal{H}} \left| \mathbb{P}_{x \sim D_S}[h(x) = 1] - \mathbb{P}_{x \sim D_T}[h(x) = 1] \right|.$$

L'idée est la suivante :

- si aucun classifieur de $\mathcal{H}$ ne distingue bien les points source des points target, alors les deux domaines sont proches relativement à $\mathcal{H}$ ;

- si un classifieur de $\mathcal{H}$ sépare facilement source et target, alors les deux domaines sont éloignés.

Le papier montre que cette divergence peut être estimée à partir d'échantillons non étiquetés des deux domaines en entraînant un classifieur de domaine. On donne aux points source le label 0, et aux points target le label 1. La capacité du modèle à distinguer les deux domaines permet d'estimer la divergence.

### 6.1 III.1 — Définition : classe d'hypothèses utilisée pour mesurer la divergence

Dans notre expérience, nous utilisons une régression logistique sur la transformation :

$$\varphi(x) = (x_1, x_2, \sin(2x_1)) .$$

Notre classe d'hypothèses est donc la classe des classifieurs linéaires dans l'espace transformé :

$$\mathcal{H} = \{x \mapsto \mathbf{1}[w^\top \varphi(x) + b \geq 0]\} .$$

Comme $\varphi(x) \in \mathbb{R}^3$, un classifieur affine dans cet espace possède une VC-dimension :

$$d = 3 + 1 = 4.$$

Ainsi, dans la suite, on fixe :

$$\mathrm{VCdim}(\mathcal{H}) = 4.$$

Cette classe est symétrique : si $h \in \mathcal{H}$, alors $1 - h \in \mathcal{H}$, car il suffit de remplacer $(w, b)$ par $(-w, -b)$.

[18]:
```python
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import numpy as np


def make_balanced_unlabeled_samples(
    X_source: np.ndarray,
    X_target: np.ndarray,
    random_state: int = 42,
) -> tuple[np.ndarray, np.ndarray]:
    """
    Construit deux échantillons non étiquetés de même taille.

    Le lemme 1 suppose que U et U' ont la même taille m.
    On prend donc m = min(n_source, n_target).
```

```
    Parameters
    ----------
    X_source : np.ndarray
        Observations du domaine source.
    X_target : np.ndarray
        Observations du domaine target.
    random_state : int
        Graine aléatoire.

    Returns
    -------
    tuple[np.ndarray, np.ndarray]
        Échantillons source et target de même taille.
    """

    rng = np.random.default_rng(random_state)

    m = min(len(X_source), len(X_target))

    source_indices = rng.choice(len(X_source), size=m, replace=False)
    target_indices = rng.choice(len(X_target), size=m, replace=False)

    U_source = X_source[source_indices]
    U_target = X_target[target_indices]

    return U_source, U_target


U_source, U_target = make_balanced_unlabeled_samples(
    X_source=source.X,
    X_target=target.X,
    random_state=42,
)

m_unlabeled = len(U_source)

print("Taille des échantillons non étiquetés")
print("-" * 50)
print(f"m = {m_unlabeled}")
print(f"U_source shape = {U_source.shape}")
print(f"U_target shape = {U_target.shape}")
```

```
Taille des échantillons non étiquetés
--------------------------------------------------
m = 300
U_source shape = (300, 2)
U_target shape = (300, 2)
```

### 6.2 III.2 — Estimation empirique de la $\mathcal{H}$-divergence

Pour estimer la divergence, on construit une tâche auxiliaire :

$$z = \begin{cases} 0 & \text{si } x \sim D_S, \\ 1 & \text{si } x \sim D_T. \end{cases}$$

On entraîne ensuite un classifieur de domaine $g \in \mathcal{H}$ pour prédire si un point vient du domaine source ou du domaine target.

Si l'erreur du meilleur classifieur de domaine est notée $\hat{\epsilon}_{dom}$, alors on estime :

$$\hat{d}_{\mathcal{H}}(U_S, U_T) = 2(1 - 2\hat{\epsilon}_{dom}).$$

Interprétation :

- si $\hat{\epsilon}_{dom} \approx 0.5$, le classifieur ne distingue pas les domaines, donc

$$\hat{d}_{\mathcal{H}}(U_S, U_T) \approx 0;$$

- si $\hat{\epsilon}_{dom} \approx 0$, le classifieur sépare presque parfaitement les domaines, donc

$$\hat{d}_{\mathcal{H}}(U_S, U_T) \approx 2.$$

Comme notre régression logistique ne résout pas exactement le problème de minimisation sur toute la classe $\mathcal{H}$, cette quantité doit être vue comme une approximation empirique de la $\mathcal{H}$-divergence.

[19]:
```python
def estimate_empirical_h_divergence(
    U_source: np.ndarray,
    U_target: np.ndarray,
) -> dict:
    """
    Estime la H-divergence empirique à partir d'un classifieur de domaine.

    Les points source reçoivent le label 0.
    Les points target reçoivent le label 1.

    On entraîne une régression logistique dans le même espace de représentation
    que celui utilisé pour l'hypothèse h_S.

    Parameters
    ----------
    U_source : np.ndarray
        Échantillon non étiqueté issu de D_S.
    U_target : np.ndarray
        Échantillon non étiqueté issu de D_T.

    Returns
```

```
    -------
    dict
        Résultats contenant l'erreur de domaine et la divergence empirique.
    """

    X_domain = np.vstack([U_source, U_target])

    y_domain = np.concatenate(
        [
            np.zeros(len(U_source), dtype=int),
            np.ones(len(U_target), dtype=int),
        ]
    )

    X_domain_phi = feature_map(X_domain)

    domain_classifier = Pipeline(
        steps=[
            ("scaler", StandardScaler()),
            ("classifier", LogisticRegression(max_iter=1000)),
        ]
    )

    domain_classifier.fit(X_domain_phi, y_domain)

    y_domain_pred = domain_classifier.predict(X_domain_phi)

    domain_accuracy = accuracy_score(y_domain, y_domain_pred)
    domain_error = 1.0 - domain_accuracy

    # Formule empirique : d_H = 2(1 - 2 * erreur)
    empirical_h_divergence = 2.0 * (1.0 - 2.0 * domain_error)

    # Par sécurité numérique, on contraint la divergence dans [0, 2].
    empirical_h_divergence = np.clip(empirical_h_divergence, 0.0, 2.0)

    return {
        "domain_classifier": domain_classifier,
        "domain_accuracy": domain_accuracy,
        "domain_error": domain_error,
        "empirical_h_divergence": empirical_h_divergence,
    }


h_divergence_results = estimate_empirical_h_divergence(
    U_source=U_source,
    U_target=U_target,
```

```
)

domain_accuracy = h_divergence_results["domain_accuracy"]
domain_error = h_divergence_results["domain_error"]
empirical_h_divergence = h_divergence_results["empirical_h_divergence"]

print("Estimation empirique de la H-divergence")
print("-" * 55)
print(f"Accuracy du classifieur de domaine       = {domain_accuracy:.4f}")
print(f"Erreur du classifieur de domaine         = {domain_error:.4f}")
print(f"d_H empirique estimée                    = {empirical_h_divergence:.
  ↪4f}")
```

```
Estimation empirique de la H-divergence
-------------------------------------------------------
Accuracy du classifieur de domaine       = 0.7217
Erreur du classifieur de domaine         = 0.2783
d_H empirique estimée                    = 0.8867
```

## 6.3 III.3 — Lemme 1 : divergence empirique et estimation à partir d'échantillons

On cherche maintenant à passer de la divergence théorique

$$d_{\mathcal{H}}(D_S, D_T) = 2 \sup_{h \in \mathcal{H}} \left| \mathbb{P}_{D_S}[h(x) = 1] - \mathbb{P}_{D_T}[h(x) = 1] \right|$$

à une quantité calculable à partir de deux échantillons non étiquetés.

Soient

$$U = \{x_1, \dots, x_m\} \sim D_S, \qquad U' = \{x'_1, \dots, x'_m\} \sim D_T.$$

On remplace les probabilités vraies par des fréquences empiriques :

$$\widehat{\Pr}_U[h(x) = 1] = \frac{1}{m} \sum_{x \in U} \mathbf{1}[h(x) = 1],$$

et

$$\widehat{\Pr}_{U'}[h(x) = 1] = \frac{1}{m} \sum_{x \in U'} \mathbf{1}[h(x) = 1].$$

### 6.3.1 Énoncé du Lemme 1

La divergence empirique associée à la classe $\mathcal{H}$ est :

$$\hat{d}_{\mathcal{H}}(U, U') = 2 \sup_{h \in \mathcal{H}} \left| \widehat{\Pr}_U[h(x) = 1] - \widehat{\Pr}_{U'}[h(x) = 1] \right|.$$

### 6.3.2 Démonstration simple

La preuve consiste simplement à remplacer, dans la définition de la vraie divergence, les probabilités théoriques par leurs versions empiriques.

Dans la vraie divergence, on compare :

$$\mathbb{P}_{D_S}[h(x) = 1] \quad \text{et} \quad \mathbb{P}_{D_T}[h(x) = 1].$$

Comme $D_S$ et $D_T$ sont inconnues, on approxime ces quantités par :

$$\frac{1}{m} \sum_{x \in U} \mathbf{1}[h(x) = 1] \quad \text{et} \quad \frac{1}{m} \sum_{x \in U'} \mathbf{1}[h(x) = 1].$$

En remplaçant dans la définition, on obtient directement :

$$\hat{d}_{\mathcal{H}}(U, U') = 2 \sup_{h \in \mathcal{H}} \left| \frac{1}{m} \sum_{x \in U} \mathbf{1}[h(x) = 1] - \frac{1}{m} \sum_{x \in U'} \mathbf{1}[h(x) = 1] \right|.$$

Le facteur 2 est conservé afin que la divergence appartienne à l'intervalle $[0, 2]$.

[20]:
```python
def empirical_positive_rate(model: Pipeline, X: np.ndarray) -> float:
    """
    Calcule la fréquence empirique des prédictions positives h(x)=1.

    Ici, le modèle de domaine a été entraîné dans l'espace feature_map(X).
    """
    y_pred = model.predict(feature_map(X))
    return float(np.mean(y_pred == 1))


p_source_positive = empirical_positive_rate(
    h_divergence_results["domain_classifier"],
    U_source,
)

p_target_positive = empirical_positive_rate(
    h_divergence_results["domain_classifier"],
    U_target,
)

h_gap_for_trained_classifier = abs(p_source_positive - p_target_positive)
h_divergence_for_trained_classifier = 2.0 * h_gap_for_trained_classifier

lemma_1_table = pd.DataFrame({
    "quantité": [
        "Pr_U[h(x)=1]",
        "Pr_U'[h(x)=1]",
```

```
        "|Pr_U - Pr_U'|",
        "2 |Pr_U - Pr_U'|",
    ],
    "valeur empirique": [
        p_source_positive,
        p_target_positive,
        h_gap_for_trained_classifier,
        h_divergence_for_trained_classifier,
    ],
})

lemma_1_table.round(4)
```

[20]:

| | quantité | valeur empirique |
|---|---|---|
| 0 | Pr_U[h(x)=1] | 0.2967 |
| 1 | Pr_U'[h(x)=1] | 0.7400 |
| 2 | \|Pr_U - Pr_U'\| | 0.4433 |
| 3 | 2 \|Pr_U - Pr_U'\| | 0.8867 |

Le tableau précédent applique le Lemme 1 au classifieur de domaine entraîné dans notre expérience.

Attention : la formule du Lemme 1 contient un supremum sur toute la classe $\mathcal{H}$. En pratique, on ne parcourt pas exactement toutes les hypothèses possibles ; on utilise le classifieur appris comme approximation du meilleur séparateur source/cible.

## 6.4 III.4 — Lemme 2 : calcul pratique de $\hat{d}_{\mathcal{H}}$ par classification binaire

Le Lemme 1 donne une expression empirique, mais elle contient encore un supremum sur toutes les hypothèses de $\mathcal{H}$.

Le Lemme 2 transforme cette quantité en un problème plus pratique : entraîner un classifieur binaire qui distingue les données source des données target.

### 6.4.1 Hypothèse de symétrie

On suppose que la classe $\mathcal{H}$ est symétrique :

$$h \in \mathcal{H} \quad \Longrightarrow \quad 1 - h \in \mathcal{H}.$$

Cela signifie que si un classifieur appartient à $\mathcal{H}$, alors son classifieur inverse appartient aussi à $\mathcal{H}$.

Dans notre cas, cette hypothèse est naturelle : inverser un classifieur linéaire revient à changer le signe de ses poids.

### 6.4.2 Énoncé du Lemme 2

On construit un jeu de classification de domaine :

$$y = 0 \quad \text{pour les points de } U,$$

et

$$y = 1 \quad \text{pour les points de } U'.$$

Pour une hypothèse $h \in \mathcal{H}$, son erreur empirique de classification de domaine est :

$$\hat{\varepsilon}_{dom}(h) = \frac{1}{2m} \left( \sum_{x \in U} \mathbf{1}[h(x) = 1] + \sum_{x \in U'} \mathbf{1}[h(x) = 0] \right).$$

Le premier terme compte les erreurs faites sur les points source, car ils devraient être prédits comme 0.
Le second terme compte les erreurs faites sur les points target, car ils devraient être prédits comme 1.

Alors :

$$\hat{d}_{\mathcal{H}}(U, U') = 2 \left( 1 - 2 \min_{h \in \mathcal{H}} \hat{\varepsilon}_{dom}(h) \right).$$

Cette formule dit que plus le meilleur classifieur de domaine a une erreur faible, plus la divergence est grande.

## 6.5 III.5 — Démonstration du Lemme 2

On note :

$$I(h) = \{x : h(x) = 1\}.$$

Autrement dit, $I(h)$ est l'ensemble des points que l'hypothèse $h$ classe comme appartenant au domaine target.

Les fréquences empiriques associées sont :

$$\widehat{\Pr}_U[I(h)] = \frac{1}{m} \sum_{x \in U} \mathbf{1}[h(x) = 1],$$

et

$$\widehat{\Pr}_{U'}[I(h)] = \frac{1}{m} \sum_{x \in U'} \mathbf{1}[h(x) = 1].$$

### 6.5.1 Étape 1 — Écriture de l'erreur de domaine

Par définition :

$$\hat{\varepsilon}_{dom}(h) = \frac{1}{2m} \left( \sum_{x \in U} \mathbf{1}[h(x) = 1] + \sum_{x \in U'} \mathbf{1}[h(x) = 0] \right).$$

Or :

$$\mathbf{1}[h(x) = 0] = 1 - \mathbf{1}[h(x) = 1].$$

Donc :

$$\frac{1}{m} \sum_{x \in U'} \mathbf{1}[h(x) = 0] = 1 - \frac{1}{m} \sum_{x \in U'} \mathbf{1}[h(x) = 1].$$

Ainsi :

$$\hat{\varepsilon}_{dom}(h) = \frac{1}{2} \left( \widehat{\Pr}_U[I(h)] + 1 - \widehat{\Pr}_{U'}[I(h)] \right).$$

Donc :

$$\hat{\varepsilon}_{dom}(h) = \frac{1}{2} \left( 1 + \widehat{\Pr}_U[I(h)] - \widehat{\Pr}_{U'}[I(h)] \right).$$

### 6.5.2 Étape 2 — Réorganisation des termes

On multiplie par 2 :

$$2\hat{\varepsilon}_{dom}(h) = 1 + \widehat{\Pr}_U[I(h)] - \widehat{\Pr}_{U'}[I(h)].$$

Donc :

$$\widehat{\Pr}_U[I(h)] - \widehat{\Pr}_{U'}[I(h)] = 2\hat{\varepsilon}_{dom}(h) - 1.$$

En prenant la valeur absolue :

$$\left| \widehat{\Pr}_U[I(h)] - \widehat{\Pr}_{U'}[I(h)] \right| = \left| 2\hat{\varepsilon}_{dom}(h) - 1 \right|.$$

La valeur absolue est essentielle : elle permet de mesurer l'écart entre les domaines sans dépendre du sens dans lequel le classifieur les sépare.

### 6.5.3 Étape 3 — Utilisation de la symétrie de $\mathcal{H}$

Si $h$ sépare les domaines dans le mauvais sens, alors $1 - h$ les sépare dans le bon sens.

Comme $\mathcal{H}$ est symétrique, $1 - h \in \mathcal{H}$. On peut donc choisir, entre $h$ et son inverse, celui qui donne l'erreur de domaine la plus petite.

Ainsi, pour le meilleur classifieur de domaine, on peut écrire :

$$\sup_{h \in \mathcal{H}} \left| \widehat{\Pr}_U[I(h)] - \widehat{\Pr}_{U'}[I(h)] \right| = 1 - 2 \min_{h \in \mathcal{H}} \hat{\varepsilon}_{dom}(h).$$

### 6.5.4 Étape 4 — Conclusion

Par le Lemme 1 :

$$\hat{d}_{\mathcal{H}}(U, U') = 2 \sup_{h \in \mathcal{H}} \left| \widehat{\Pr}_U[I(h)] - \widehat{\Pr}_{U'}[I(h)] \right|.$$

En remplaçant le supremum par l'expression obtenue à l'étape précédente, on obtient :

$$\boxed{\hat{d}_{\mathcal{H}}(U, U') = 2 \left( 1 - 2 \min_{h \in \mathcal{H}} \hat{\varepsilon}_{dom}(h) \right)}$$

ce qui démontre le Lemme 2.

[21]:
```python
def empirical_domain_error_from_predictions(
    pred_source: np.ndarray,
    pred_target: np.ndarray,
) -> float:
    """
    Calcule l'erreur empirique de domaine pour la convention :
    - label 0 pour la source ;
    - label 1 pour la target.
    """
    source_errors = np.mean(pred_source == 1)
    target_errors = np.mean(pred_target == 0)
    return 0.5 * (source_errors + target_errors)


domain_model = h_divergence_results["domain_classifier"]

pred_source = domain_model.predict(feature_map(U_source))
pred_target = domain_model.predict(feature_map(U_target))

error_h = empirical_domain_error_from_predictions(pred_source, pred_target)
error_inverse_h = empirical_domain_error_from_predictions(1 - pred_source, 1 - pred_target)
error_symmetric = min(error_h, error_inverse_h)

lemma_2_divergence = 2.0 * (1.0 - 2.0 * error_symmetric)
lemma_2_divergence = float(np.clip(lemma_2_divergence, 0.0, 2.0))

lemma_2_table = pd.DataFrame({
    "quantité": [
        "erreur de h",
        "erreur de 1-h",
        "min(erreur de h, erreur de 1-h)",
        "2(1 - 2 erreur_min)",
    ],
```

```
    "valeur": [
        error_h,
        error_inverse_h,
        error_symmetric,
        lemma_2_divergence,
    ],
})

lemma_2_table.round(4)
```

[21]:

| | quantité | valeur |
|---|---|---|
| 0 | erreur de h | 0.2783 |
| 1 | erreur de 1-h | 0.7217 |
| 2 | min(erreur de h, erreur de 1-h) | 0.2783 |
| 3 | 2(1 - 2 erreur_min) | 0.8867 |

Le tableau illustre le rôle de la symétrie : on compare l'erreur de $h$ et celle de $1-h$, puis on garde la plus petite. La divergence estimée par le Lemme 2 est alors :

$$2(1-2\hat{\varepsilon}_{dom}).$$

Dans la suite, cette estimation est utilisée comme approximation pratique de $\hat{d}_{\mathcal{H}}(U,U')$.

## 6.6 III.6 — Proposition : borne de concentration sur la vraie $\mathcal{H}$-divergence

La divergence empirique est calculée à partir d'échantillons finis. Il faut donc contrôler l'écart entre :

$$\hat{d}_{\mathcal{H}}(U,U')$$

et la vraie divergence :

$$d_{\mathcal{H}}(D_S,D_T).$$

### 6.6.1 Énoncé de la proposition

Soient $U$ et $U'$ deux échantillons de taille $m$, tirés respectivement selon $D$ et $D'$. Si $\mathcal{H}$ a une VC-dimension $d$, alors pour tout $\delta \in (0,1)$, avec probabilité au moins $1-\delta$ :

$$d_{\mathcal{H}}(D,D') \leq \hat{d}_{\mathcal{H}}(U,U') + 4\sqrt{\frac{d\log(2m)+\log\left(\frac{2}{\delta}\right)}{m}}.$$

Dans notre expérience :

$$D = D_S, \qquad D' = D_T, \qquad \mathrm{VCdim}(\mathcal{H}) = 4.$$

Cette proposition relie donc la quantité observée sur les échantillons à la vraie divergence entre distributions.

[22]:
```
def h_divergence_upper_bound(
    empirical_h_divergence: float,
    vc_dimension: int,
    m: int,
    delta: float,
) -> dict:
    """
    Calcule la borne supérieure sur la vraie H-divergence.

    Formule :
        d_H(D,D') <= d_hat_H(U,U')
                     + 4 sqrt((d log(2m) + log(2/delta)) / m)
    """
    concentration_term = 4.0 * np.sqrt(
        (vc_dimension * np.log(2.0 * m) + np.log(2.0 / delta)) / m
    )

    raw_upper_bound = empirical_h_divergence + concentration_term
    clipped_upper_bound = min(raw_upper_bound, 2.0)

    return {
        "empirical_h_divergence": empirical_h_divergence,
        "concentration_term": concentration_term,
        "raw_upper_bound": raw_upper_bound,
        "clipped_upper_bound": clipped_upper_bound,
    }


vc_dimension_H = 4
delta = 0.05

h_bound_results = h_divergence_upper_bound(
    empirical_h_divergence=empirical_h_divergence,
    vc_dimension=vc_dimension_H,
    m=m_unlabeled,
    delta=delta,
)

pd.DataFrame({
    "quantité": [
        "d_H empirique",
        "terme de concentration",
        "borne brute",
        "borne tronquée dans [0,2]",
```

```
    ],
    "valeur": [
        h_bound_results["empirical_h_divergence"],
        h_bound_results["concentration_term"],
        h_bound_results["raw_upper_bound"],
        h_bound_results["clipped_upper_bound"],
    ],
}).round(4)
```

[22]:

| | quantité | valeur |
|---|---|---|
| 0 | d_H empirique | 0.8867 |
| 1 | terme de concentration | 1.2496 |
| 2 | borne brute | 2.1362 |
| 3 | borne tronquée dans [0,2] | 2.0000 |

[23]:
```
def plot_h_divergence_summary(
    empirical_h_divergence: float,
    concentration_term: float,
    clipped_upper_bound: float,
) -> None:
    """
    Affiche la H-divergence empirique et sa borne supérieure.
    """
    names = [
        "d_H empirique",
        "Terme de concentration",
        "Borne sur d_H",
    ]

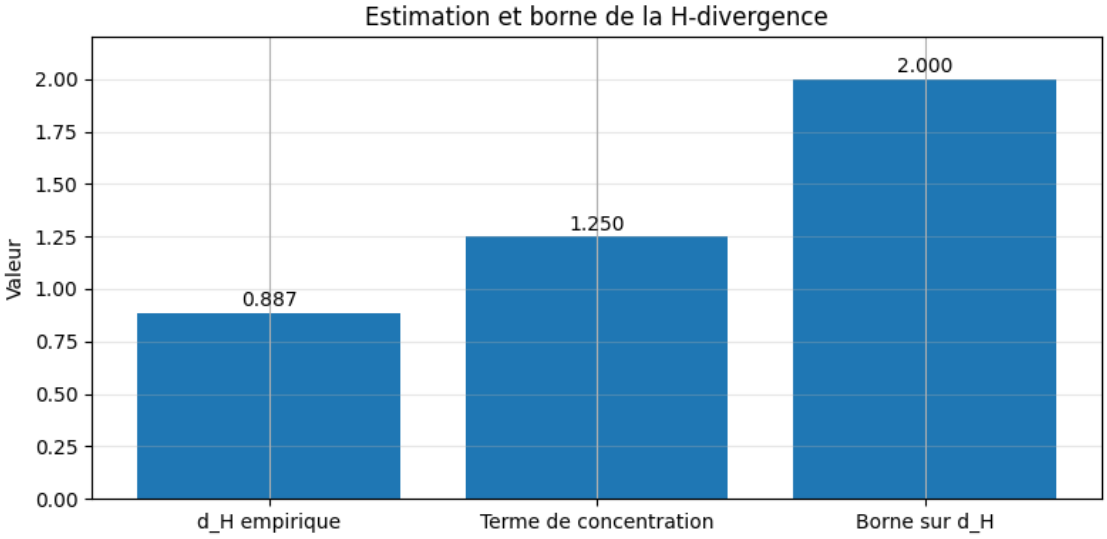
    values = [
        empirical_h_divergence,
        concentration_term,
        clipped_upper_bound,
    ]

    plt.figure(figsize=(8, 4))
    plt.bar(names, values)
    plt.ylabel("Valeur")
    plt.title("Estimation et borne de la H-divergence")
    plt.ylim(0, max(2.1, max(values) + 0.2))
    plt.grid(axis="y", alpha=0.3)

    for i, value in enumerate(values):
        plt.text(i, value + 0.03, f"{value:.3f}", ha="center")

    plt.tight_layout()
    plt.show()
```

```
plot_h_divergence_summary(
    empirical_h_divergence=h_bound_results["empirical_h_divergence"],
    concentration_term=h_bound_results["concentration_term"],
    clipped_upper_bound=h_bound_results["clipped_upper_bound"],
)
```




## 6.7 III.7 — Interprétation de la $\mathcal{H}$-divergence

Nous avons remplacé la divergence $d_1(D_S, D_T)$, difficile à calculer, par une divergence dépendant de la classe d'hypothèses utilisée :

$$d_{\mathcal{H}}(D_S, D_T).$$

Cette divergence mesure uniquement les différences entre $D_S$ et $D_T$ qui sont visibles par les modèles de la classe $\mathcal{H}$.

Dans notre expérience, $\mathcal{H}$ est la classe des classifieurs linéaires dans l'espace transformé :

$$\varphi(x) = (x_1, x_2, \sin(2x_1)).$$

Nous avons fixé :

$$\mathrm{VCdim}(\mathcal{H}) = 4.$$

À partir d'un classifieur de domaine, nous avons obtenu une estimation empirique :

$$\hat{d}_{\mathcal{H}}(U_S, U_T).$$

Puis, grâce au lemme 1, nous avons obtenu une borne de la vraie divergence :

$$d_{\mathcal{H}}(D_S, D_T) \leq \hat{d}_{\mathcal{H}}(U_S, U_T) + 4\sqrt{\frac{d\log(2m) + \log(2/\delta)}{m}}.$$

Ce résultat est important car il rend la divergence entre domaines estimable à partir de données finies.

Cependant, pour borner directement l'erreur target d'une hypothèse $h$, le papier utilise ensuite une divergence encore plus adaptée :

$$d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T),$$

qui mesure les désaccords entre deux hypothèses de $\mathcal{H}$.

La prochaine étape consistera donc à introduire la divergence $\mathcal{H}\Delta\mathcal{H}$, puis à l'utiliser dans la borne du théorème 2.

## 6.8 III.8 — Définition : hypothèse jointe idéale et terme $\lambda$

On introduit l'hypothèse jointe idéale :

$$h^* = \arg\min_{h\in\mathcal{H}} \left[\epsilon_S(h) + \epsilon_T(h)\right].$$

Le terme associé est :

$$\lambda = \epsilon_S(h^*) + \epsilon_T(h^*).$$

Il mesure la compatibilité des deux tâches. Si $\lambda$ est faible, une même hypothèse peut bien fonctionner sur les deux domaines. Si $\lambda$ est élevé, l'adaptation est structurellement difficile.

### 6.8.1 III.8.1 — Approximation empirique de $h^*$

Dans notre expérience synthétique, les labels source et target sont connus. On peut donc approximer $h^*$ en entraînant un modèle sur les deux domaines réunis.

[24]:
```
X_joint = np.vstack([source.X, target.X])
y_joint = np.r_[source.y, target.y]

joint_model = Pipeline([
    ("features", FunctionTransformer(feature_map, validate=False)),
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=1000, random_state=42)),
])
```

```
joint_model.fit(X_joint, y_joint)

print("Modèle joint h* entraîné sur source + target.")
```

```
Modèle joint h* entraîné sur source + target.
```

### 6.8.2 III.8.2 — Calcul de $\hat{\lambda}$

On estime les deux erreurs empiriques de $h^*$ :

$$\hat{\lambda} = \hat{\epsilon}_S(h^*) + \hat{\epsilon}_T(h^*).$$

[25]:
```
def empirical_error(model: Pipeline, domain: DomainData) -> float:
    return np.mean(model.predict(domain.X) != domain.y)

lambda_source_error = empirical_error(joint_model, source)
lambda_target_error = empirical_error(joint_model, target)
lambda_hat = lambda_source_error + lambda_target_error

lambda_table = pd.DataFrame({
    "hypothèse": ["h_S", "h_S", "h*", "h*"],
    "domaine": ["source", "target", "source", "target"],
    "erreur empirique": [
        epsilon_source,
        epsilon_target,
        lambda_source_error,
        lambda_target_error,
    ],
})

display(lambda_table.round(4))
print(f"lambda estimé = {lambda_hat:.4f}")
```

| | hypothèse | domaine | erreur empirique |
|---|---|---|---|
| 0 | h_S | source | 0.0890 |
| 1 | h_S | target | 0.0833 |
| 2 | h* | source | 0.0880 |
| 3 | h* | target | 0.0733 |

```
lambda estimé = 0.1613
```

### 6.8.3 III.8.3 — Interprétation du terme $\lambda$

Le terme estimé est :

$$\hat{\lambda} = \hat{\epsilon}_S(h^*) + \hat{\epsilon}_T(h^*).$$

Il mesure la meilleure erreur jointe que l'on peut espérer obtenir dans notre classe d'hypothèses.

Dans notre cas, $\mathcal{H}$ est la classe des classifieurs linéaires dans l'espace transformé :

$$\varphi(x) = (x_1, x_2, \sin(2x_1)).$$

Si $\hat{\lambda}$ est faible, alors notre classe d'hypothèses contient un modèle capable de s'adapter raisonnablement aux deux domaines.

Si $\hat{\lambda}$ est élevé, alors le problème d'adaptation est difficile, non seulement à cause du décalage entre $D_S$ et $D_T$, mais aussi parce que les fonctions d'étiquetage $f_S$ et $f_T$ ne sont pas bien compatibles dans la classe $\mathcal{H}$.

### 6.8.4 III.8.4 — Bilan sur $\lambda$

Le terme $\lambda$ complète l'analyse : il ne compare pas seulement les distributions, mais aussi les fonctions d'étiquetage.

Dans notre exemple, $f_S$ et $f_T$ sont proches mais non identiques. Le terme $\hat{\lambda}$ permet donc de quantifier empiriquement le coût minimal d'une hypothèse commune aux deux domaines.

## 6.9 III.9 — Définition : espace de désaccord $\mathcal{H}\Delta\mathcal{H}$

Pour borner l'erreur target, on ne compare plus seulement les distributions $D_S$ et $D_T$. On compare aussi la façon dont les hypothèses de $\mathcal{H}$ se désaccordent sur ces deux domaines.

On définit :

$$\mathcal{H}\Delta\mathcal{H} = \{g : g(x) = h(x) \oplus h'(x),\ h, h' \in \mathcal{H}\}.$$

Ainsi, $g(x) = 1$ exactement lorsque $h(x) \neq h'(x)$. La divergence associée est :

$$d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) = 2 \sup_{h,h' \in \mathcal{H}} \left| \mathbb{P}_{D_S}[h(x) \neq h'(x)] - \mathbb{P}_{D_T}[h(x) \neq h'(x)] \right|.$$

Dans le notebook, le supremum est remplacé par un maximum sur un ensemble fini d'hypothèses candidates.

### 6.9.1 III.9.1 — Construction d'hypothèses candidates

Pour obtenir plusieurs hypothèses dans $\mathcal{H}$, on entraîne des modèles avec différents poids entre source et target :

$$(1-\alpha)\hat{\epsilon}_S(h) + \alpha\hat{\epsilon}_T(h), \qquad \alpha \in [0,1].$$

Quand $\alpha = 0$, le modèle est surtout source. Quand $\alpha = 1$, il est surtout target.

[26]:
```
def empirical_classification_error(
    y_true: np.ndarray,
    y_pred: np.ndarray,
) -> float:
```

```
    """
    Calcule l'erreur de classification empirique moyenne.
    """
    return float(np.mean(y_true != y_pred))
```

[27]:
```
def train_mixed_model(alpha: float) -> Pipeline:
    X = np.vstack([source.X, target.X])
    y = np.r_[source.y, target.y]

    n_s, n_t = len(source.X), len(target.X)
    weights = np.r_[
        np.full(n_s, (1 - alpha) / n_s),
        np.full(n_t, alpha / n_t),
    ]

    model = Pipeline([
        ("features", FunctionTransformer(feature_map, validate=False)),
        ("scaler", StandardScaler()),
        ("classifier", LogisticRegression(max_iter=1000, random_state=42)),
    ])
    model.fit(X, y, classifier__sample_weight=weights)
    return model

alpha_grid = np.linspace(0, 1, 11)
candidate_models = [train_mixed_model(alpha) for alpha in alpha_grid]

pd.DataFrame({"alpha": alpha_grid}).T
```

[27]:

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| alpha | 0.0 | 0.1 | 0.2 | 0.3 | 0.4 | 0.5 | 0.6 | 0.7 | 0.8 | 0.9 | 1.0 |

### 6.9.2 III.9.2 — Estimation empirique de $d_{\mathcal{H}\Delta\mathcal{H}}$

Pour deux hypothèses $h_i$ et $h_j$, on estime :

$$\hat{\epsilon}_S(h_i, h_j) = \frac{1}{n_S} \sum_{x \in S} \mathbf{1}[h_i(x) \neq h_j(x)]$$

et de même sur le domaine target. On garde ensuite la paire qui maximise l'écart entre les deux désaccords.

[28]:
```
def disagreement_error(model_a: Pipeline, model_b: Pipeline, X: np.ndarray) ->␣
 ↪float:
    return np.mean(model_a.predict(X) != model_b.predict(X))

rows = []
for i, model_i in enumerate(candidate_models):
```

```
    for j, model_j in enumerate(candidate_models[i + 1:], start=i + 1):
        eps_s = disagreement_error(model_i, model_j, source.X)
        eps_t = disagreement_error(model_i, model_j, target.X)
        rows.append({
            "alpha_i": alpha_grid[i],
            "alpha_j": alpha_grid[j],
            "epsilon_S": eps_s,
            "epsilon_T": eps_t,
            "écart": abs(eps_s - eps_t),
        })

disagreement_table = pd.DataFrame(rows)
best_pair = disagreement_table.loc[disagreement_table["écart"].idxmax()]
empirical_h_delta_h = 2 * best_pair["écart"]

print(f"d_HΔH empirique approximée = {empirical_h_delta_h:.4f}")
display(best_pair.to_frame("valeur").T.round(4))
```

```
d_HΔH empirique approximée = 0.1807
```

| | alpha_i | alpha_j | epsilon_S | epsilon_T | écart |
|---|---|---|---|---|---|
| valeur | 0.0 | 1.0 | 0.127 | 0.0367 | 0.0903 |

### 6.9.3 III 9.3 Pourquoi passer de la $\mathcal{H}$-divergence à la $\mathcal{H}\Delta\mathcal{H}$-divergence ?

La $\mathcal{H}$-divergence mesure la capacité d'une hypothèse $h \in \mathcal{H}$ à distinguer les exemples provenant du domaine source de ceux provenant du domaine cible.

Elle est définie par :

$$d_{\mathcal{H}}(D_S, D_T) = 2 \sup_{h \in \mathcal{H}} \left| \Pr_{x \sim D_S}[h(x) = 1] - \Pr_{x \sim D_T}[h(x) = 1] \right|.$$

Cette quantité est utile car elle indique si les deux distributions $D_S$ et $D_T$ sont proches ou non du point de vue de la classe d'hypothèses $\mathcal{H}$.

Cependant, dans les bornes d'adaptation de domaine, on ne cherche pas seulement à savoir si une hypothèse peut distinguer les domaines.
On cherche surtout à contrôler la différence d'erreur d'une même hypothèse entre la source et la cible :

$$|\varepsilon_S(h, h') - \varepsilon_T(h, h')|.$$

Autrement dit, on veut comparer la manière dont deux hypothèses $h$ et $h'$ sont en désaccord sur le domaine source et sur le domaine cible.

---

**Intuition** L'erreur d'une hypothèse $h$ dépend de la région de l'espace où elle se trompe.

Mais cette région d'erreur peut être décrite comme une région de désaccord entre deux hypothèses :

$$h(x) \neq h'(x).$$

Dans les preuves d'adaptation de domaine, on compare souvent une hypothèse apprise $h$ à une hypothèse idéale $h^*$, par exemple celle qui minimise simultanément l'erreur source et l'erreur cible.

Il faut donc contrôler des quantités du type :

$$\Pr_{x \sim D_S}[h(x) \neq h^*(x)]$$

et

$$\Pr_{x \sim D_T}[h(x) \neq h^*(x)].$$

La $\mathcal{H}$-divergence classique ne suffit pas directement, car elle regarde seulement les ensembles de la forme :

$$\{x : h(x) = 1\}.$$

Or ici, on doit regarder les ensembles de désaccord :

$$\{x : h(x) \neq h'(x)\}.$$

C'est précisément pour cela qu'on introduit la classe :

$$\mathcal{H}\Delta\mathcal{H} = \{x \mapsto h(x) \oplus h'(x) : h, h' \in \mathcal{H}\},$$

où $\oplus$ désigne le XOR.

Une fonction de $\mathcal{H}\Delta\mathcal{H}$ vaut 1 exactement lorsque deux hypothèses de $\mathcal{H}$ sont en désaccord.

---

**Définition** La $\mathcal{H}\Delta\mathcal{H}$-divergence est donc définie par :

$$d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) = 2 \sup_{h,h' \in \mathcal{H}} \left| \Pr_{x \sim D_S}[h(x) \neq h'(x)] - \Pr_{x \sim D_T}[h(x) \neq h'(x)] \right|.$$

Elle mesure à quel point les régions de désaccord entre hypothèses changent lorsqu'on passe du domaine source au domaine cible.

---

**Pourquoi c'est la bonne divergence pour l'adaptation de domaine ?** La quantité centrale dans la preuve du théorème d'adaptation est :

$$|\varepsilon_S(h, h') - \varepsilon_T(h, h')|\,.$$

Or, pour une perte 0-1, on a :

$$\varepsilon_S(h, h') = \Pr_{x \sim D_S}[h(x) \neq h'(x)],$$

et :

$$\varepsilon_T(h, h') = \Pr_{x \sim D_T}[h(x) \neq h'(x)].$$

Donc :

$$|\varepsilon_S(h, h') - \varepsilon_T(h, h')| = \left| \Pr_{x \sim D_S}[h(x) \neq h'(x)] - \Pr_{x \sim D_T}[h(x) \neq h'(x)] \right|.$$

Par définition de la $\mathcal{H}\Delta\mathcal{H}$-divergence, cette quantité est bornée par :

$$\frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T).$$

Ainsi :

$$\boxed{|\varepsilon_S(h, h') - \varepsilon_T(h, h')| \leq \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T)}$$

---

**Conclusion intuitive** On passe de la $\mathcal{H}$-divergence à la $\mathcal{H}\Delta\mathcal{H}$-divergence parce que les bornes d'adaptation de domaine ne contrôlent pas seulement la capacité à distinguer source et cible.

Elles doivent contrôler la stabilité des désaccords entre hypothèses lorsqu'on change de domaine.

La $\mathcal{H}$-divergence regarde les régions où une hypothèse prédit 1.
La $\mathcal{H}\Delta\mathcal{H}$-divergence regarde les régions où deux hypothèses sont en désaccord.

C'est cette seconde notion qui est adaptée aux preuves de généralisation, car l'erreur entre deux hypothèses est exactement une probabilité de désaccord.

### 6.9.4 III.9.4 — Lemme 3 : contrôle du changement de désaccord

Le lemme 3 affirme que, pour tous $h, h' \in \mathcal{H}$ :

$$|\epsilon_S(h, h') - \epsilon_T(h, h')| \leq \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T).$$

On le vérifie ici pour la paire $(h_S, h^*)$.

[29]:
```
eps_s_hs_hstar = disagreement_error(source_model, joint_model, source.X)
eps_t_hs_hstar = disagreement_error(source_model, joint_model, target.X)
lemma_left = abs(eps_s_hs_hstar - eps_t_hs_hstar)
lemma_right = 0.5 * empirical_h_delta_h

lemma_table = pd.DataFrame({
    "quantité": [
        "epsilon_S(h_S,h*)",
        "epsilon_T(h_S,h*)",
        "membre gauche",
        "1/2 d_HΔH empirique",
    ],
    "valeur": [eps_s_hs_hstar, eps_t_hs_hstar, lemma_left, lemma_right],
})

display(lemma_table.round(4))
print("Inégalité vérifiée :", lemma_left <= lemma_right + 1e-12)
```

```
              quantité  valeur
0    epsilon_S(h_S,h*)  0.0070
1    epsilon_T(h_S,h*)  0.0233
2        membre gauche  0.0163
3  1/2 d_HΔH empirique  0.0903
```

```
Inégalité vérifiée : True
```

### 6.9.5 III.9.5 — Bilan avant le Théorème 2

La divergence $\mathcal{H}\Delta\mathcal{H}$ mesure un changement plus fin que la simple séparation des domaines : elle regarde si les zones de désaccord entre hypothèses restent stables entre source et target.

Le lemme 3 montre que cette divergence contrôle le changement de désaccord entre deux hypothèses. Elle permet donc de passer de l'erreur source à l'erreur target dans la borne d'adaptation.

La suite naturelle est le théorème de borne target :

$$\epsilon_T(h) \leq \epsilon_S(h) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) + \lambda.$$

## 6.10 III.10 — Introduction du Théorème 2 : borne sur l'erreur target

Les parties précédentes ont introduit trois quantités importantes :

$$\epsilon_S(h), \qquad d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T), \qquad \lambda.$$

Le théorème 2 relie ces trois termes à l'erreur target d'une hypothèse $h$.

L'idée est la suivante : si une hypothèse a une faible erreur sur le domaine source, si les deux domaines sont proches au sens de la divergence $\mathcal{H}\Delta\mathcal{H}$, et si le terme $\lambda$ est faible, alors son erreur sur le domaine target est contrôlée.

### 6.10.1 III.10.1 — Théorème 2

Soit $\mathcal{H}$ une classe d'hypothèses de VC-dimension $d$.

Soient $U_S$ et $U_T$ deux échantillons non étiquetés de taille $m'$, tirés respectivement selon $D_S$ et $D_T$.

Alors, pour tout $\delta \in (0,1)$, avec probabilité au moins $1-\delta$, pour toute hypothèse $h \in \mathcal{H}$, on a :

$$\epsilon_T(h) \leq \epsilon_S(h) + \frac{1}{2}\hat{d}_{\mathcal{H}\Delta\mathcal{H}}(U_S, U_T) + 4\sqrt{\frac{2d\log(2m') + \log\left(\frac{2}{\delta}\right)}{m'}} + \lambda.$$

Cette borne contient quatre termes :

$$\epsilon_S(h)$$

mesure la performance de $h$ sur le domaine source.

$$\frac{1}{2}\hat{d}_{\mathcal{H}\Delta\mathcal{H}}(U_S, U_T)$$

mesure l'écart entre les domaines source et target.

$$4\sqrt{\frac{2d\log(2m') + \log\left(\frac{2}{\delta}\right)}{m'}}$$

est un terme de concentration statistique.

Enfin,

$$\lambda = \epsilon_S(h^*) + \epsilon_T(h^*)$$

mesure l'erreur de la meilleure hypothèse jointe sur les deux domaines.

### 6.10.2 III.10.2 — Interprétation du Théorème 2

Le théorème donne une borne de la forme :

$$\epsilon_T(h) \leq \text{erreur source} + \text{écart entre domaines} + \text{terme statistique} + \text{erreur jointe idéale}.$$

Donc l'adaptation de domaine est favorable lorsque :

- $\epsilon_S(h)$ est faible ;
- la divergence $\mathcal{H}\Delta\mathcal{H}$ est faible ;
- le nombre d'exemples non étiquetés $m'$ est grand ;

- $\lambda$ est faible.

Le terme $\lambda$ est important : si aucune hypothèse de $\mathcal{H}$ ne fonctionne bien à la fois sur source et target, alors l'adaptation devient difficile, même si la divergence entre domaines est faible.

[30]:
```python
def theorem_2_bound(
    epsilon_source: float,
    empirical_h_delta_h: float,
    lambda_value: float,
    vc_dimension: int,
    m_unlabeled: int,
    delta: float,
) -> dict:
    concentration = 4.0 * np.sqrt(
        (
            2.0 * vc_dimension * np.log(2.0 * m_unlabeled)
            + np.log(2.0 / delta)
        )
        / m_unlabeled
    )

    bound = (
        epsilon_source
        + 0.5 * empirical_h_delta_h
        + concentration
        + lambda_value
    )

    return {
        "epsilon_source": epsilon_source,
        "half_h_delta_h": 0.5 * empirical_h_delta_h,
        "concentration": concentration,
        "lambda": lambda_value,
        "bound": bound,
    }
```

### 6.10.3 III.10.3 — Application du Théorème 2 à l'hypothèse $h_S$

On applique maintenant le théorème 2 à l'hypothèse $h_S$, c'est-à-dire au modèle entraîné sur le domaine source.

Dans notre notebook :

$$\hat{\epsilon}_S(h_S)$$

est l'erreur empirique de $h_S$ sur le domaine source.

$$\hat{d}_{\mathcal{H}\Delta\mathcal{H}}(U_S, U_T)$$

est l'approximation empirique de la divergence construite dans la partie précédente.

Enfin, on remplace $\lambda$ par son approximation empirique :

$$\hat{\lambda} = \hat{\epsilon}_S(h^*) + \hat{\epsilon}_T(h^*).$$

[31]:
```python
def vc_dimension_linear_proxy(X: np.ndarray) -> int:
    return feature_map(X).shape[1] + 1
```

[32]:
```python
delta = 0.05
m_unlabeled = min(len(source.X), len(target.X))

vc_dimension_proxy = vc_dimension_linear_proxy(source.X)

theorem_2_results = theorem_2_bound(
    epsilon_source=epsilon_source,
    empirical_h_delta_h=empirical_h_delta_h,
    lambda_value=lambda_hat,
    vc_dimension=vc_dimension_proxy,
    m_unlabeled=m_unlabeled,
    delta=delta,
)

theorem_2_table = pd.DataFrame(
    [
        {
            "Terme": "Erreur source",
            "Notation": "epsilon_S(h_S)",
            "Valeur": theorem_2_results["epsilon_source"],
        },
        {
            "Terme": "Divergence",
            "Notation": "0.5 d_HΔH",
            "Valeur": theorem_2_results["half_h_delta_h"],
        },
        {
            "Terme": "Concentration",
            "Notation": "terme statistique",
            "Valeur": theorem_2_results["concentration"],
        },
        {
            "Terme": "Erreur jointe idéale",
            "Notation": "lambda",
            "Valeur": theorem_2_results["lambda"],
        },
        {
            "Terme": "Borne finale",
```

```
            "Notation": "borne sur epsilon_T(h_S)",
            "Valeur": theorem_2_results["bound"],
        },
    ]
)

theorem_2_table
```

[32]:

| | Terme | Notation | Valeur |
|---|---|---|---|
| 0 | Erreur source | epsilon_S(h_S) | 0.089000 |
| 1 | Divergence | 0.5 d_HΔH | 0.090333 |
| 2 | Concentration | terme statistique | 1.710584 |
| 3 | Erreur jointe idéale | lambda | 0.161333 |
| 4 | Borne finale | borne sur epsilon_T(h_S) | 2.051250 |

[33]:
```
comparison_theorem_2 = pd.DataFrame(
    [
        {
            "Quantité": "Erreur target observée",
            "Valeur": epsilon_target,
        },
        {
            "Quantité": "Borne du théorème 2",
            "Valeur": theorem_2_results["bound"],
        },
    ]
)

comparison_theorem_2
```

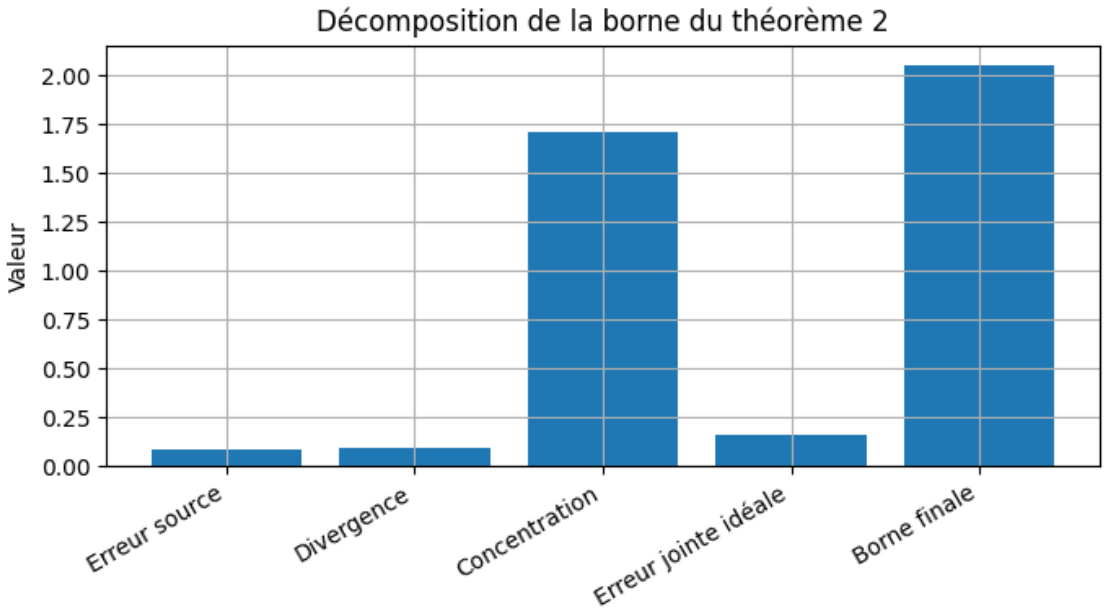
[33]:

| | Quantité | Valeur |
|---|---|---|
| 0 | Erreur target observée | 0.083333 |
| 1 | Borne du théorème 2 | 2.051250 |

[34]:
```
plt.figure(figsize=(7, 4))

plt.bar(
    theorem_2_table["Terme"],
    theorem_2_table["Valeur"],
)

plt.xticks(rotation=30, ha="right")
plt.ylabel("Valeur")
plt.title("Décomposition de la borne du théorème 2")
plt.tight_layout()
plt.show()
```




### 6.10.4 III.10.4 — Idée de la démonstration du Théorème 2

La preuve repose sur deux ingrédients.

Le premier est l'inégalité triangulaire pour l'erreur de classification. Pour trois fonctions $f_1$, $f_2$ et $f_3$, on a :

$$\epsilon(f_1, f_2) \leq \epsilon(f_1, f_3) + \epsilon(f_2, f_3).$$

On l'applique à $h$, à l'hypothèse jointe idéale $h^*$, et à la fonction d'étiquetage du domaine target.

On obtient d'abord :

$$\epsilon_T(h) \leq \epsilon_T(h^*) + \epsilon_T(h, h^*).$$

Le lemme 3 permet ensuite de contrôler le passage de $D_T$ vers $D_S$ :

$$\epsilon_T(h, h^*) \leq \epsilon_S(h, h^*) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T).$$

Puis on applique encore l'inégalité triangulaire sur le domaine source :

$$\epsilon_S(h, h^*) \leq \epsilon_S(h) + \epsilon_S(h^*).$$

En combinant ces trois inégalités, on obtient :

$$\epsilon_T(h) \leq \epsilon_S(h) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) + \epsilon_S(h^*) + \epsilon_T(h^*).$$

Comme

$$\lambda = \epsilon_S(h^*) + \epsilon_T(h^*),$$

on a :

$$\epsilon_T(h) \leq \epsilon_S(h) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) + \lambda.$$

Enfin, la divergence vraie est remplacée par sa version empirique, avec un terme de concentration :

$$d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) \leq \hat{d}_{\mathcal{H}\Delta\mathcal{H}}(U_S, U_T) + 8\sqrt{\frac{2d\log(2m') + \log\left(\frac{2}{\delta}\right)}{m'}}.$$

Comme cette divergence est multipliée par $\frac{1}{2}$, le terme ajouté dans la borne finale devient :

$$4\sqrt{\frac{2d\log(2m') + \log\left(\frac{2}{\delta}\right)}{m'}}.$$

### 6.10.5 III.10.5 — Conclusion de la Partie III

Le théorème 2 donne une justification théorique à l'adaptation de domaine.

Il montre que l'erreur target d'une hypothèse $h$ peut être contrôlée par :

$$\epsilon_S(h) + \frac{1}{2}\hat{d}_{\mathcal{H}\Delta\mathcal{H}}(U_S, U_T) + \text{terme de concentration} + \lambda.$$

Ainsi, entraîner un bon modèle sur le domaine source ne suffit pas toujours.

Il faut aussi que les domaines soient proches au sens de la divergence $\mathcal{H}\Delta\mathcal{H}$, et qu'il existe une hypothèse capable d'être bonne à la fois sur source et target.

## 6.11 Conclusion après le Théorème 2

Le Théorème 2 constitue le résultat central de l'adaptation de domaine étudiée dans ce rapport. Il remplace la distance directe entre distributions par une quantité plus adaptée à l'apprentissage : la divergence $\mathcal{H}\Delta\mathcal{H}$.

La forme générale de la borne est :

$$\varepsilon_T(h) \leq \varepsilon_S(h) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) + \lambda.$$

Chaque terme possède une interprétation précise.

Le terme $\varepsilon_S(h)$ mesure la qualité du modèle sur le domaine source. Le terme $d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T)$ mesure à quel point les régions de désaccord entre hypothèses changent lorsqu'on passe du domaine source au domaine cible. Enfin, le terme $\lambda$ mesure l'existence d'une hypothèse capable d'être bonne simultanément sur les deux domaines.

Cette borne est plus informative que celle du Théorème 1, car elle relie directement l'erreur cible à des objets liés à la classe de modèles utilisée. Elle montre aussi une limite fondamentale : même si la divergence entre domaines est faible, l'adaptation peut rester difficile si $\lambda$ est grand, c'est-à-dire si aucune hypothèse de $\mathcal{H}$ ne convient bien aux deux domaines à la fois.

# 7 Partie IV — Du Théorème 2 au Théorème 3 : apprentissage source-target pondéré

Le théorème 2 permet de relier l'erreur target d'une hypothèse à son erreur source, à la divergence entre domaines et au terme $\lambda$.

On s'intéresse maintenant à une situation plus générale : le modèle n'est plus entraîné uniquement sur le domaine source, mais sur un mélange de données source et target.

Le learner reçoit un échantillon :

$$S = (S_T, S_S)$$

de taille totale $m$, où :

$$|S_T| = \beta m$$

et

$$|S_S| = (1 - \beta)m.$$

Le paramètre $\beta \in [0, 1]$ décrit donc la proportion de données target disponibles pendant l'entraînement.

## 7.1 IV.1 — Définition : erreur empirique pondérée

L'objectif final reste de minimiser l'erreur target :

$$\epsilon_T(h).$$

Mais lorsque $\beta$ est petit, il y a peu de données target disponibles. Minimiser uniquement l'erreur empirique target peut alors être instable.

On introduit donc une erreur empirique pondérée :

$$\hat{\epsilon}_\alpha(h) = \alpha\hat{\epsilon}_T(h) + (1 - \alpha)\hat{\epsilon}_S(h),$$

où $\alpha \in [0, 1]$.

Le paramètre $\alpha$ contrôle l'importance donnée au domaine target dans le critère d'entraînement.

De même, on définit l'erreur vraie pondérée :

$$\epsilon_\alpha(h) = \alpha\epsilon_T(h) + (1 - \alpha)\epsilon_S(h).$$

Ainsi :

- si $\alpha = 0$, on entraîne uniquement sur le domaine source ;
- si $\alpha = 1$, on entraîne uniquement sur le domaine target ;
- si $0 < \alpha < 1$, on combine source et target.

[35]:
```python
def weighted_error(
    epsilon_source_value: float,
    epsilon_target_value: float,
    alpha: float,
) -> float:
    return (
        alpha * epsilon_target_value
        + (1.0 - alpha) * epsilon_source_value
    )
```

[36]:
```python
def train_alpha_model(
    X_source: np.ndarray,
    y_source: np.ndarray,
    X_target: np.ndarray,
    y_target: np.ndarray,
    alpha: float,
) -> Pipeline:
    X = np.vstack([X_source, X_target])
    y = np.concatenate([y_source, y_target])

    n_source = len(X_source)
    n_target = len(X_target)

    weights = np.concatenate([
        np.full(n_source, (1.0 - alpha) / n_source),
        np.full(n_target, alpha / n_target),
    ])

    model = Pipeline([
        ("scaler", StandardScaler()),
        ("classifier", LogisticRegression(max_iter=1000)),
    ])

    model.fit(
        feature_map(X),
```

```
        y,
        classifier__sample_weight=weights,
    )

    return model
```

## 7.2 IV.2 — Lemme 4 : lien entre erreur pondérée et erreur target

Pour toute hypothèse $h \in \mathcal{H}$, on a :

$$|\epsilon_\alpha(h) - \epsilon_T(h)| \leq (1-\alpha)\left[\frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) + \lambda\right].$$

Ce lemme montre que l'erreur pondérée $\epsilon_\alpha(h)$ est proche de l'erreur target lorsque $\alpha$ est proche de 1.

En effet, si $\alpha = 1$, alors :

$$\epsilon_\alpha(h) = \epsilon_T(h),$$

et le membre de droite devient nul.

Au contraire, si $\alpha$ est petit, l'erreur pondérée dépend davantage du domaine source. Dans ce cas, l'écart entre source et target intervient à travers la divergence $\mathcal{H}\Delta\mathcal{H}$ et le terme $\lambda$.

### 7.2.1 IV.3 Démonstration du lemme 4

On rappelle que l'erreur pondérée est définie par :

$$\epsilon_\alpha(h) = \alpha\epsilon_T(h) + (1-\alpha)\epsilon_S(h).$$

Le lemme 4 affirme que, pour toute hypothèse $h \in \mathcal{H}$,

$$|\epsilon_\alpha(h) - \epsilon_T(h)| \leq (1-\alpha)\left[\frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) + \lambda\right].$$

---

**Preuve** On part de la différence entre l'erreur pondérée et l'erreur target :

$$|\epsilon_\alpha(h) - \epsilon_T(h)|.$$

En remplaçant $\epsilon_\alpha(h)$ par sa définition, on obtient :

$$|\alpha\epsilon_T(h) + (1-\alpha)\epsilon_S(h) - \epsilon_T(h)|.$$

On regroupe les termes en $\epsilon_T(h)$ :

$$|(1-\alpha)\epsilon_S(h) - (1-\alpha)\epsilon_T(h)|\,.$$

Donc :

$$|\epsilon_\alpha(h) - \epsilon_T(h)| = (1-\alpha)\,|\epsilon_S(h) - \epsilon_T(h)|\,.$$

Il reste donc à contrôler l'écart entre l'erreur source et l'erreur target de la même hypothèse $h$.

---

**Introduction de l'hypothèse jointe idéale** On introduit l'hypothèse jointe idéale :

$$h^* = \arg\min_{h\in\mathcal{H}} [\epsilon_S(h) + \epsilon_T(h)]\,.$$

On rappelle aussi que :

$$\lambda = \epsilon_S(h^*) + \epsilon_T(h^*).$$

On ajoute et on retranche les termes de désaccord avec $h^*$ :

$$|\epsilon_S(h) - \epsilon_T(h)| \leq |\epsilon_S(h) - \epsilon_S(h,h^*)| + |\epsilon_S(h,h^*) - \epsilon_T(h,h^*)| + |\epsilon_T(h,h^*) - \epsilon_T(h)|\,.$$

Cette étape est simplement l'inégalité triangulaire.

---

**Contrôle des trois termes** Pour le premier terme, sur le domaine source, la différence entre l'erreur de $h$ et le désaccord entre $h$ et $h^*$ est contrôlée par l'erreur de $h^*$ :

$$|\epsilon_S(h) - \epsilon_S(h,h^*)| \leq \epsilon_S(h^*).$$

De même, sur le domaine target :

$$|\epsilon_T(h,h^*) - \epsilon_T(h)| \leq \epsilon_T(h^*).$$

Le terme central mesure le changement de désaccord entre source et target :

$$|\epsilon_S(h,h^*) - \epsilon_T(h,h^*)|\,.$$

Par définition de la $\mathcal{H}\Delta\mathcal{H}$-divergence, comme $h$ et $h^*$ appartiennent à $\mathcal{H}$, on a :

$$|\epsilon_S(h,h^*) - \epsilon_T(h,h^*)| \leq \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T).$$

En combinant ces trois contrôles, on obtient :

$$|\epsilon_S(h) - \epsilon_T(h)| \leq \epsilon_S(h^*) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) + \epsilon_T(h^*).$$

Or :

$$\lambda = \epsilon_S(h^*) + \epsilon_T(h^*).$$

Donc :

$$|\epsilon_S(h) - \epsilon_T(h)| \leq \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) + \lambda.$$

Enfin, comme :

$$|\epsilon_\alpha(h) - \epsilon_T(h)| = (1 - \alpha) \, |\epsilon_S(h) - \epsilon_T(h)| \, ,$$

on obtient :

$$\boxed{|\epsilon_\alpha(h) - \epsilon_T(h)| \leq (1 - \alpha) \left[ \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) + \lambda \right]}$$

Ce qui démontre le lemme 4.

[37]:
```python
def lemma_4_bound(
    alpha: float,
    empirical_h_delta_h: float,
    lambda_value: float,
) -> float:
    return (1.0 - alpha) * (
        0.5 * empirical_h_delta_h + lambda_value
    )
```

[38]:
```python
alpha_grid = np.linspace(0.0, 1.0, 11)

lemma_4_rows = []

for alpha in alpha_grid:
    model = train_alpha_model(
        X_source=source.X,
        y_source=source.y,
        X_target=target.X,
        y_target=target.y,
        alpha=alpha,
    )
```

```
    y_pred_source = model.predict(feature_map(source.X))
    y_pred_target = model.predict(feature_map(target.X))

    eps_s = empirical_classification_error(source.y, y_pred_source)
    eps_t = empirical_classification_error(target.y, y_pred_target)
    eps_alpha = weighted_error(eps_s, eps_t, alpha)

    lemma_4_rows.append({
        "alpha": alpha,
        "epsilon_S": eps_s,
        "epsilon_T": eps_t,
        "epsilon_alpha": eps_alpha,
        "|epsilon_alpha - epsilon_T|": abs(eps_alpha - eps_t),
        "borne lemme 4": lemma_4_bound(
            alpha=alpha,
            empirical_h_delta_h=empirical_h_delta_h,
            lambda_value=lambda_hat,
        ),
    })

lemma_4_table = pd.DataFrame(lemma_4_rows)
lemma_4_table
```

[38]:

| | alpha | epsilon_S | … | \|epsilon_alpha - epsilon_T\| | borne lemme 4 |
|---|---|---|---|---|---|
| 0 | 0.0 | 0.096 | … | 0.029333 | 0.251667 |
| 1 | 0.1 | 0.107 | … | 0.036300 | 0.226500 |
| 2 | 0.2 | 0.108 | … | 0.033067 | 0.201333 |
| 3 | 0.3 | 0.108 | … | 0.031267 | 0.176167 |
| 4 | 0.4 | 0.106 | … | 0.027600 | 0.151000 |
| 5 | 0.5 | 0.112 | … | 0.024333 | 0.125833 |
| 6 | 0.6 | 0.123 | … | 0.021200 | 0.100667 |
| 7 | 0.7 | 0.126 | … | 0.016800 | 0.075500 |
| 8 | 0.8 | 0.132 | … | 0.011733 | 0.050333 |
| 9 | 0.9 | 0.150 | … | 0.007333 | 0.025167 |
| 10 | 1.0 | 0.169 | … | 0.000000 | 0.000000 |

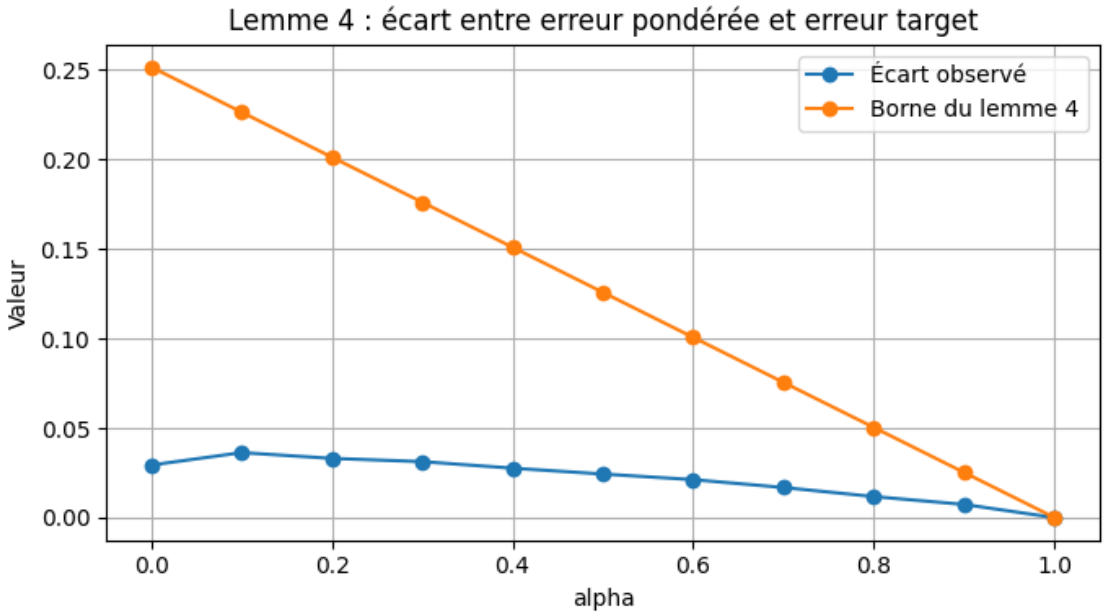
[11 rows x 6 columns]

[39]:
```
plt.figure(figsize=(7, 4))

plt.plot(
    lemma_4_table["alpha"],
    lemma_4_table["|epsilon_alpha - epsilon_T|"],
    marker="o",
    label="Écart observé",
)
```

```
plt.plot(
    lemma_4_table["alpha"],
    lemma_4_table["borne lemme 4"],
    marker="o",
    label="Borne du lemme 4",
)

plt.xlabel("alpha")
plt.ylabel("Valeur")
plt.title("Lemme 4 : écart entre erreur pondérée et erreur target")
plt.legend()
plt.tight_layout()
plt.show()
```




### 7.3 IV.4 — Interprétation du Lemme 4

Le tableau précédent compare deux quantités.

La première est l'écart observé :

$$|\hat{\epsilon}_\alpha(h) - \hat{\epsilon}_T(h)|\,.$$

La seconde est la borne donnée par le lemme 4 :

$$(1-\alpha)\left[\frac{1}{2}\hat{d}_{\mathcal{H}\Delta\mathcal{H}} + \hat{\lambda}\right].$$

Lorsque $\alpha$ augmente, l'entraînement accorde plus de poids aux données target.

On observe alors que la borne diminue, car le facteur $(1-\alpha)$ devient plus petit.

Cela correspond bien à l'intuition : plus on utilise de données target, moins l'écart entre source et target influence le critère d'apprentissage.

### 7.4 IV.5 — Lemme 5 : concentration de l'erreur pondérée

Le lemme 5 contrôle l'écart entre l'erreur pondérée vraie et son estimation empirique.

Pour une hypothèse fixée $h$, si l'échantillon de taille $m$ contient :

$$\beta m$$

points target et

$$(1-\beta)m$$

points source, alors pour tout $\delta \in (0,1)$, avec probabilité au moins $1-\delta$ :

$$\mathbb{P}\left(|\hat{\epsilon}_\alpha(h) - \epsilon_\alpha(h)| \geq \varepsilon\right) \leq 2\exp\left(\frac{-2m\varepsilon^2}{\frac{\alpha^2}{\beta} + \frac{(1-\alpha)^2}{1-\beta}}\right).$$

Le dénominateur

$$\frac{\alpha^2}{\beta} + \frac{(1-\alpha)^2}{1-\beta}$$

montre que la concentration dépend à la fois :

- du poids $\alpha$ choisi dans le critère d'apprentissage ;
- de la proportion réelle $\beta$ de données target disponibles.

[40]:
```python
def lemma_5_concentration(
    alpha: float,
    beta: float,
    m: int,
    delta: float,
) -> float:
    variance_factor = (
        alpha**2 / beta
        + (1.0 - alpha)**2 / (1.0 - beta)
    )

    return np.sqrt(
        variance_factor * np.log(2.0 / delta) / (2.0 * m)
    )
```

[41]:
```
delta = 0.05

m_total = len(source.X) + len(target.X)
beta = len(target.X) / m_total

concentration_rows = []

for alpha in alpha_grid:
    concentration_rows.append({
        "alpha": alpha,
        "beta": beta,
        "concentration lemme 5": lemma_5_concentration(
            alpha=alpha,
            beta=beta,
            m=m_total,
            delta=delta,
        ),
    })

lemma_5_table = pd.DataFrame(concentration_rows)
lemma_5_table
```

[41]:

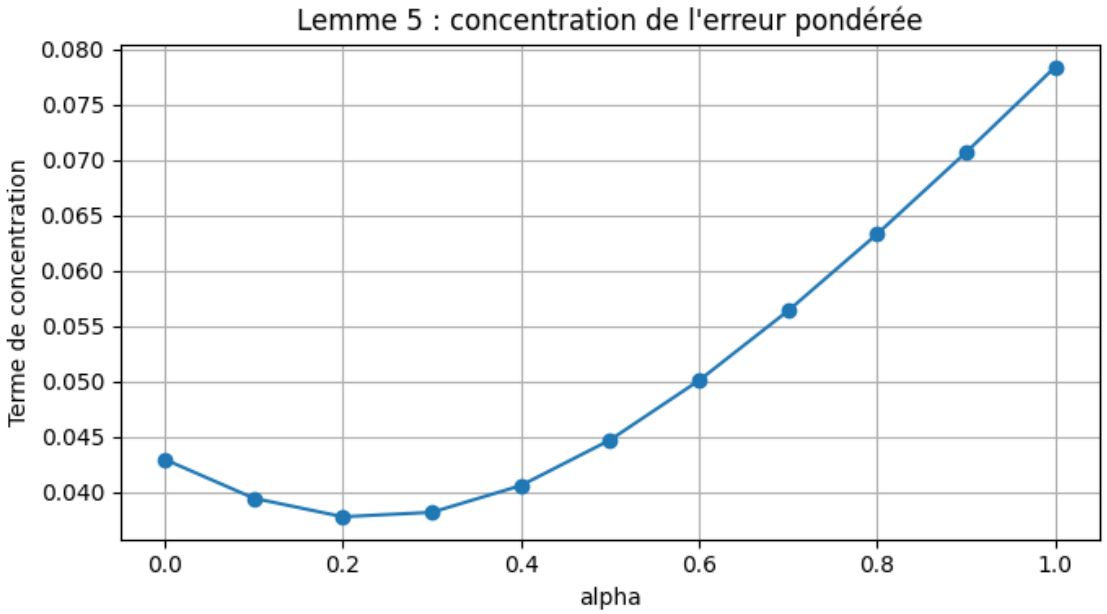
| | alpha | beta | concentration lemme 5 |
|---|---|---|---|
| 0 | 0.0 | 0.230769 | 0.042947 |
| 1 | 0.1 | 0.230769 | 0.039440 |
| 2 | 0.2 | 0.230769 | 0.037767 |
| 3 | 0.3 | 0.230769 | 0.038172 |
| 4 | 0.4 | 0.230769 | 0.040592 |
| 5 | 0.5 | 0.230769 | 0.044701 |
| 6 | 0.6 | 0.230769 | 0.050084 |
| 7 | 0.7 | 0.230769 | 0.056379 |
| 8 | 0.8 | 0.230769 | 0.063313 |
| 9 | 0.9 | 0.230769 | 0.070700 |
| 10 | 1.0 | 0.230769 | 0.078410 |

[42]:
```
plt.figure(figsize=(7, 4))

plt.plot(
    lemma_5_table["alpha"],
    lemma_5_table["concentration lemme 5"],
    marker="o",
)

plt.xlabel("alpha")
plt.ylabel("Terme de concentration")
plt.title("Lemme 5 : concentration de l'erreur pondérée")
plt.tight_layout()
```

```
plt.show()
```




### 7.5 IV.6 — Rôle des paramètres $\alpha$ et $\beta$

Les paramètres $\alpha$ et $\beta$ ont des rôles différents.

Le paramètre $\beta$ décrit les données disponibles :

$$\beta = \frac{\text{nombre de points target}}{\text{nombre total de points}}.$$

Il est donc imposé par le problème.

Le paramètre $\alpha$, lui, est choisi par l'algorithme d'apprentissage :

$$\hat{\epsilon}_\alpha(h) = \alpha\hat{\epsilon}_T(h) + (1-\alpha)\hat{\epsilon}_S(h).$$

Lorsque $\alpha$ est proche de $\beta$, le critère d'apprentissage reflète la proportion réelle des données.

Mais en adaptation de domaine, on peut choisir $\alpha$ différent de $\beta$ pour donner plus ou moins d'importance au domaine target.

### 7.6 IV.7 — Conclusion intermédiaire

Cette partie introduit l'apprentissage à partir d'un mélange de données source et target.

Le critère étudié est :

$$\hat{\epsilon}_\alpha(h) = \alpha\hat{\epsilon}_T(h) + (1-\alpha)\hat{\epsilon}_S(h).$$

Le lemme 4 montre que l'écart entre l'erreur pondérée et l'erreur target est contrôlé par :

$$(1-\alpha)\left[\frac{1}{2}d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) + \lambda\right].$$

Le lemme 5 montre que l'erreur pondérée empirique se concentre autour de l'erreur pondérée vraie, avec un terme dépendant de $\alpha$, $\beta$, $m$ et $\delta$.

Ces deux résultats préparent une borne d'apprentissage pour les algorithmes qui minimisent une combinaison d'erreur source et target.

## 7.7 IV.8 — Proposition 1 : inégalité de Hoeffding

On rappelle d'abord l'inégalité de Hoeffding.

Soient $X_1, \dots, X_n$ des variables aléatoires indépendantes telles que :

$$a_i \leq X_i \leq b_i$$

pour tout $i \in \{1, \dots, n\}$.

On note :

$$\overline{X} = \frac{1}{n}\sum_{i=1}^{n} X_i.$$

Alors, pour tout $\varepsilon > 0$,

$$\mathbb{P}\left(\left|\overline{X} - \mathbb{E}[\overline{X}]\right| \geq \varepsilon\right) \leq 2\exp\left(\frac{-2n^2\varepsilon^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right).$$

Cette inégalité permet de contrôler l'écart entre une moyenne empirique et son espérance.

## 7.8 IV.8 — Démonstration du Lemme 5

On veut contrôler l'écart :

$$\left|\hat{\epsilon}_\alpha(h) - \epsilon_\alpha(h)\right|.$$

L'échantillon total contient $m$ observations :

- $\beta m$ observations target dans $S_T$ ;
- $(1-\beta)m$ observations source dans $S_S$.

Pour les observations target, on définit :

$$X_i = \frac{\alpha}{\beta} \left| h(x_i) - f_T(x_i) \right|, \qquad i = 1, \dots, \beta m.$$

Pour les observations source, on définit :

$$X_i = \frac{1-\alpha}{1-\beta} \left| h(x_i) - f_S(x_i) \right|, \qquad i = \beta m + 1, \dots, m.$$

Comme les erreurs de classification valent 0 ou 1, on a :

$$X_i \in \left[0, \frac{\alpha}{\beta}\right]$$

pour les observations target, et :

$$X_i \in \left[0, \frac{1-\alpha}{1-\beta}\right]$$

pour les observations source.

### 7.8.1 IV.8.1 — Réécriture de $\hat{\epsilon}_\alpha(h)$

Par définition :

$$\hat{\epsilon}_\alpha(h) = \alpha \hat{\epsilon}_T(h) + (1-\alpha)\hat{\epsilon}_S(h).$$

Or :

$$\hat{\epsilon}_T(h) = \frac{1}{\beta m} \sum_{x \in S_T} |h(x) - f_T(x)|$$

et :

$$\hat{\epsilon}_S(h) = \frac{1}{(1-\beta)m} \sum_{x \in S_S} \left| h(x) - f_S(x) \right|.$$

Donc :

$$\hat{\epsilon}_\alpha(h) = \frac{1}{m} \sum_{i=1}^{m} X_i.$$

Autrement dit, l'erreur empirique pondérée est une moyenne empirique des variables $X_i$.

### 7.8.2 IV.8.2 — Calcul de l'espérance

Par linéarité de l'espérance :

$$\mathbb{E}\left[\widehat{\epsilon}_\alpha(h)\right] = \alpha\epsilon_T(h) + (1-\alpha)\epsilon_S(h).$$

Donc :

$$\mathbb{E}\left[\widehat{\epsilon}_\alpha(h)\right] = \epsilon_\alpha(h).$$

Ainsi, contrôler :

$$|\widehat{\epsilon}_\alpha(h) - \epsilon_\alpha(h)|$$

revient à contrôler l'écart entre la moyenne empirique des $X_i$ et son espérance.

### 7.8.3 IV.8.3 — Application de l'inégalité de Hoeffding

On applique l'inégalité de Hoeffding aux variables indépendantes $X_1, \dots, X_m$.

Pour les $\beta m$ variables target, l'amplitude est :

$$\frac{\alpha}{\beta}.$$

Pour les $(1-\beta)m$ variables source, l'amplitude est :

$$\frac{1-\alpha}{1-\beta}.$$

Donc :

$$\sum_{i=1}^{m}(b_i - a_i)^2 = \beta m\left(\frac{\alpha}{\beta}\right)^2 + (1-\beta)m\left(\frac{1-\alpha}{1-\beta}\right)^2.$$

On simplifie :

$$\sum_{i=1}^{m}(b_i - a_i)^2 = m\left[\frac{\alpha^2}{\beta} + \frac{(1-\alpha)^2}{1-\beta}\right].$$

En remplaçant dans Hoeffding, on obtient :

$$\mathbb{P}\left(|\widehat{\epsilon}_\alpha(h) - \epsilon_\alpha(h)| \geq \varepsilon\right) \leq 2\exp\left(\frac{-2m\varepsilon^2}{\frac{\alpha^2}{\beta} + \frac{(1-\alpha)^2}{1-\beta}}\right).$$

C'est exactement le lemme 5.

### 7.8.4 IV.8.4 — Interprétation de la concentration

Le lemme 5 montre que la fiabilité de l'estimation empirique dépend du facteur :

$$\frac{\alpha^2}{\beta} + \frac{(1-\alpha)^2}{1-\beta}.$$

Ce facteur est minimal lorsque $\alpha$ est proche de $\beta$.

Cela signifie que l'estimation empirique est plus stable lorsque le poids donné aux données target correspond à leur proportion réelle dans l'échantillon.

À l'inverse, si $\alpha$ est très différent de $\beta$, certaines observations reçoivent un poids plus important, ce qui rend l'estimation moins fiable.

[43]:
```
def hoeffding_weight_factor(alpha: float, beta: float) -> float:
    return (
        alpha**2 / beta
        + (1.0 - alpha)**2 / (1.0 - beta)
    )
```

[44]:
```
beta = len(target.X) / (len(source.X) + len(target.X))
alpha_grid = np.linspace(0.0, 1.0, 11)

hoeffding_rows = []

for alpha in alpha_grid:
    hoeffding_rows.append({
        "alpha": alpha,
        "beta": beta,
        "facteur": hoeffding_weight_factor(alpha, beta),
    })

hoeffding_table = pd.DataFrame(hoeffding_rows)
hoeffding_table
```

[44]:

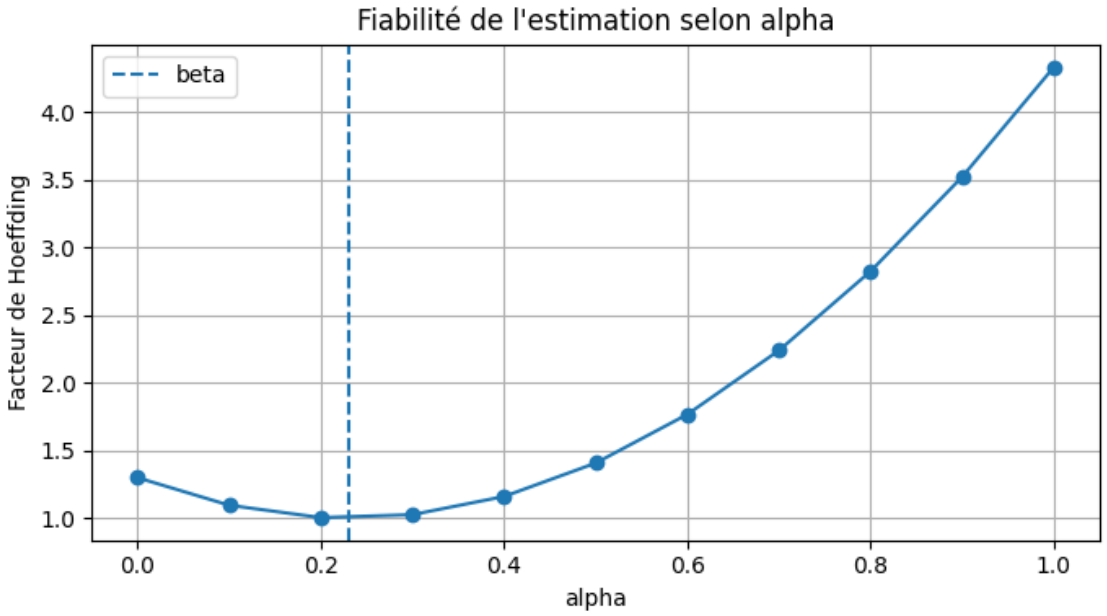
| | alpha | beta | facteur |
|---|---|---|---|
| 0 | 0.0 | 0.230769 | 1.300000 |
| 1 | 0.1 | 0.230769 | 1.096333 |
| 2 | 0.2 | 0.230769 | 1.005333 |
| 3 | 0.3 | 0.230769 | 1.027000 |
| 4 | 0.4 | 0.230769 | 1.161333 |
| 5 | 0.5 | 0.230769 | 1.408333 |
| 6 | 0.6 | 0.230769 | 1.768000 |
| 7 | 0.7 | 0.230769 | 2.240333 |
| 8 | 0.8 | 0.230769 | 2.825333 |
| 9 | 0.9 | 0.230769 | 3.523000 |
| 10 | 1.0 | 0.230769 | 4.333333 |

[45]:
```
plt.figure(figsize=(7, 4))

plt.plot(
    hoeffding_table["alpha"],
    hoeffding_table["facteur"],
    marker="o",
)

plt.axvline(
    beta,
    linestyle="--",
    label="beta"
)

plt.xlabel("alpha")
plt.ylabel("Facteur de Hoeffding")
plt.title("Fiabilité de l'estimation selon alpha")
plt.legend()
plt.tight_layout()
plt.show()
```




## 7.9 IV.9 — Transition vers la borne finale

Cette preuve montre que l'erreur empirique pondérée :

$$\hat{\epsilon}_\alpha(h)$$

se concentre autour de l'erreur pondérée vraie :

$$\epsilon_\alpha(h).$$

Le lemme 4 relie ensuite $\epsilon_\alpha(h)$ à l'erreur target $\epsilon_T(h)$.

En combinant ces deux résultats, on peut maintenant établir une borne d'apprentissage pour un algorithme qui minimise :

$$\hat{\epsilon}_\alpha(h).$$

## 7.10 IV.10 — Théorème 3 : borne finale avec apprentissage source-target

Le théorème 2 reliait l'erreur target d'une hypothèse à son erreur source, à la divergence entre domaines et au terme $\lambda$.

Le théorème 3 va plus loin : il considère directement un algorithme qui apprend à partir d'un mélange de données source et target.

On suppose que l'algorithme minimise l'erreur empirique pondérée :

$$\hat{\epsilon}_\alpha(h) = \alpha\hat{\epsilon}_T(h) + (1-\alpha)\hat{\epsilon}_S(h).$$

On note :

$$\hat{h} = \arg\min_{h\in\mathcal{H}} \hat{\epsilon}_\alpha(h).$$

Le but est maintenant de borner l'erreur target de cette hypothèse apprise :

$$\epsilon_T(\hat{h}).$$

### 7.10.1 IV.10.1 — Énoncé du Théorème 3

Soit $\mathcal{H}$ une classe d'hypothèses de VC-dimension $d$.

Soient $U_S$ et $U_T$ deux échantillons non étiquetés de taille $m'$, tirés respectivement selon $D_S$ et $D_T$.

Soit $S$ un échantillon étiqueté de taille $m$, contenant :

$$\beta m$$

points target et :

$$(1-\beta)m$$

points source.

On suppose que $\hat{h}$ minimise l'erreur empirique pondérée :

$$\hat{\epsilon}_\alpha(h) = \alpha\hat{\epsilon}_T(h) + (1-\alpha)\hat{\epsilon}_S(h).$$

On note aussi :

$$h_T^* = \arg\min_{h\in\mathcal{H}} \epsilon_T(h).$$

Alors, pour tout $\delta \in (0,1)$, avec probabilité au moins $1-\delta$, on a :

$$\epsilon_T(\hat{h}) \leq \epsilon_T(h_T^*) + 4\sqrt{\left(\frac{\alpha^2}{\beta} + \frac{(1-\alpha)^2}{1-\beta}\right)\frac{2d\log(2(m+1)) + 2\log\left(\frac{8}{\delta}\right)}{m}} + 2(1-\alpha)\left[\frac{1}{2}\hat{d}_{\mathcal{H}\Delta\mathcal{H}}(U_S, U_T) + 4\sqrt{\frac{2d\log(2m')}{m'}}\right.$$

[46]:
```python
def theorem_3_bound(
    epsilon_target_star: float,
    empirical_h_delta_h: float,
    lambda_value: float,
    alpha: float,
    beta: float,
    m_labeled: int,
    m_unlabeled: int,
    vc_dimension: int,
    delta: float,
) -> dict:
    weighted_complexity = (
        alpha**2 / beta
        + (1.0 - alpha)**2 / (1.0 - beta)
    )

    learning_term = 4.0 * np.sqrt(
        weighted_complexity
        * (
            2.0 * vc_dimension * np.log(2.0 * (m_labeled + 1))
            + 2.0 * np.log(8.0 / delta)
        )
        / m_labeled
    )

    divergence_concentration = 4.0 * np.sqrt(
        (
            2.0 * vc_dimension * np.log(2.0 * m_unlabeled)
            + np.log(8.0 / delta)
        )
        / m_unlabeled
    )
```

```
    adaptation_term = 2.0 * (1.0 - alpha) * (
        0.5 * empirical_h_delta_h
        + divergence_concentration
        + lambda_value
    )

    bound = (
        epsilon_target_star
        + learning_term
        + adaptation_term
    )

    return {
        "alpha": alpha,
        "epsilon_target_star": epsilon_target_star,
        "learning_term": learning_term,
        "adaptation_term": adaptation_term,
        "bound": bound,
    }
```

### 7.10.2 IV.10.2 — Approximation de $h_T^*$

Le théorème utilise :

$$h_T^* = \arg\min_{h \in \mathcal{H}} \epsilon_T(h).$$

Dans notre expérience, on ne connaît pas exactement ce minimiseur théorique.

On l'approxime donc empiriquement en entraînant un modèle uniquement sur les données target, ce qui correspond à :

$$\alpha = 1.$$

L'erreur target obtenue sert d'approximation numérique de :

$$\epsilon_T(h_T^*).$$

[47]:
```
def empirical_classification_error(
    y_true: np.ndarray,
    y_pred: np.ndarray,
) -> float:
    return np.mean(y_true != y_pred)
```

[48]:
```
target_model = train_alpha_model(
    X_source=source.X,
    y_source=source.y,
```

```
    X_target=target.X,
    y_target=target.y,
    alpha=1.0,
)

target_pred = target_model.predict(feature_map(target.X))

epsilon_target_star_hat = empirical_classification_error(
    target.y,
    target_pred,
)

epsilon_target_star_hat
```

[48]:
```
np.float64(0.08333333333333333)
```

[49]:
```
delta = 0.05

m_labeled = len(source.X) + len(target.X)
m_unlabeled = min(len(source.X), len(target.X))

beta = len(target.X) / m_labeled
vc_dimension_proxy = feature_map(source.X).shape[1] + 1

alpha_grid = np.linspace(0.0, 1.0, 11)

theorem_3_rows = []

for alpha in alpha_grid:
    model_alpha = train_alpha_model(
        X_source=source.X,
        y_source=source.y,
        X_target=target.X,
        y_target=target.y,
        alpha=alpha,
    )

    pred_source = model_alpha.predict(feature_map(source.X))
    pred_target = model_alpha.predict(feature_map(target.X))

    eps_source_alpha = empirical_classification_error(source.y, pred_source)
    eps_target_alpha = empirical_classification_error(target.y, pred_target)
    eps_alpha = weighted_error(
        eps_source_alpha,
        eps_target_alpha,
        alpha,
    )
```

```
    bound_results = theorem_3_bound(
        epsilon_target_star=epsilon_target_star_hat,
        empirical_h_delta_h=empirical_h_delta_h,
        lambda_value=lambda_hat,
        alpha=alpha,
        beta=beta,
        m_labeled=m_labeled,
        m_unlabeled=m_unlabeled,
        vc_dimension=vc_dimension_proxy,
        delta=delta,
    )

    theorem_3_rows.append({
        "alpha": alpha,
        "epsilon_S": eps_source_alpha,
        "epsilon_T observee": eps_target_alpha,
        "epsilon_alpha": eps_alpha,
        "terme apprentissage": bound_results["learning_term"],
        "terme adaptation": bound_results["adaptation_term"],
        "borne theorem 3": bound_results["bound"],
    })

theorem_3_table = pd.DataFrame(theorem_3_rows)
theorem_3_table
```

[49]:

| | alpha | epsilon_S | … | terme adaptation | borne theorem 3 |
|---|---|---|---|---|---|
| 0 | 0.0 | 0.096 | … | 3.967454 | 5.131991 |
| 1 | 0.1 | 0.107 | … | 3.570708 | 4.646946 |
| 2 | 0.2 | 0.108 | … | 3.173963 | 4.208100 |
| 3 | 0.3 | 0.108 | … | 2.777218 | 3.821546 |
| 4 | 0.4 | 0.106 | … | 2.380472 | 3.485720 |
| 5 | 0.5 | 0.112 | … | 1.983727 | 3.192413 |
| 6 | 0.6 | 0.123 | … | 1.586982 | 2.931205 |
| 7 | 0.7 | 0.126 | … | 1.190236 | 2.692929 |
| 8 | 0.8 | 0.132 | … | 0.793491 | 2.470760 |
| 9 | 0.9 | 0.150 | … | 0.396745 | 2.259965 |
| 10 | 1.0 | 0.169 | … | 0.000000 | 2.057333 |

[11 rows x 7 columns]

[50]:
```
best_theorem_3_row = theorem_3_table.loc[
    theorem_3_table["borne theorem 3"].idxmin()
]

best_theorem_3_row
```

[50]:
```
alpha                  1.000000
epsilon_S              0.169000
epsilon_T observee     0.083333
epsilon_alpha          0.083333
terme apprentissage    1.973999
terme adaptation       0.000000
borne theorem 3        2.057333
Name: 10, dtype: float64
```

### 7.10.3 IV.10.3 — Comparaison finale entre erreur observée et borne

On compare maintenant :

$$\epsilon_T(\hat{h}),$$

c'est-à-dire l'erreur target observée du modèle appris, avec la borne donnée par le théorème 3.

La borne théorique est généralement plus grande que l'erreur observée, car elle est valable uniformément avec forte probabilité et inclut plusieurs termes de sécurité statistique.

[51]:
```
final_comparison = pd.DataFrame(
    [
        {
            "Quantité": "Meilleur alpha selon la borne",
            "Valeur": best_theorem_3_row["alpha"],
        },
        {
            "Quantité": "Erreur target observée",
            "Valeur": best_theorem_3_row["epsilon_T observee"],
        },
        {
            "Quantité": "Borne du théorème 3",
            "Valeur": best_theorem_3_row["borne theorem 3"],
        },
        {
            "Quantité": "Terme d'apprentissage",
            "Valeur": best_theorem_3_row["terme apprentissage"],
        },
        {
            "Quantité": "Terme d'adaptation",
            "Valeur": best_theorem_3_row["terme adaptation"],
        },
    ]
)

final_comparison
```

[51]:

| | Quantité | Valeur |
|---|---|---|
| 0 | Meilleur alpha selon la borne | 1.000000 |
| 1 | Erreur target observée | 0.083333 |
| 2 | Borne du théorème 3 | 2.057333 |
| 3 | Terme d'apprentissage | 1.973999 |
| 4 | Terme d'adaptation | 0.000000 |

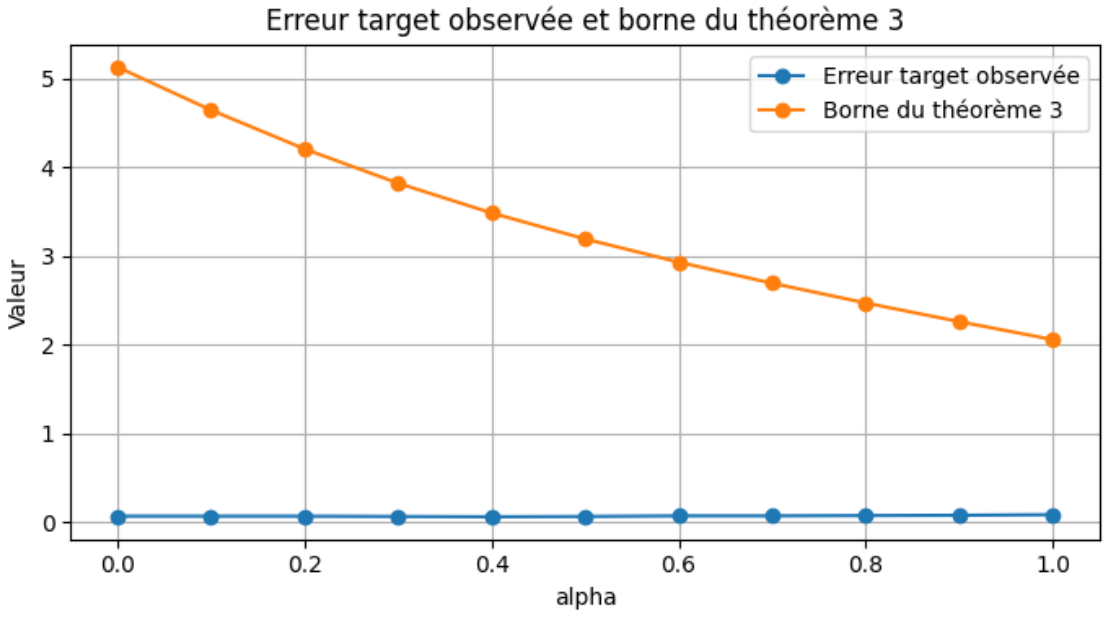
[52]:

```
plt.figure(figsize=(7, 4))

plt.plot(
    theorem_3_table["alpha"],
    theorem_3_table["epsilon_T observee"],
    marker="o",
    label="Erreur target observée",
)

plt.plot(
    theorem_3_table["alpha"],
    theorem_3_table["borne theorem 3"],
    marker="o",
    label="Borne du théorème 3",
)

plt.xlabel("alpha")
plt.ylabel("Valeur")
plt.title("Erreur target observée et borne du théorème 3")
plt.legend()
plt.tight_layout()
plt.show()
```



[53]:
```
best_alpha = best_theorem_3_row["alpha"]

decomposition_theorem_3 = pd.DataFrame(
    [
        {
            "Terme": "Erreur optimale target approximée",
            "Valeur": epsilon_target_star_hat,
        },
        {
            "Terme": "Terme d'apprentissage",
            "Valeur": best_theorem_3_row["terme apprentissage"],
        },
        {
            "Terme": "Terme d'adaptation",
            "Valeur": best_theorem_3_row["terme adaptation"],
        },
        {
            "Terme": "Borne finale",
            "Valeur": best_theorem_3_row["borne theorem 3"],
        },
    ]
)

decomposition_theorem_3
```

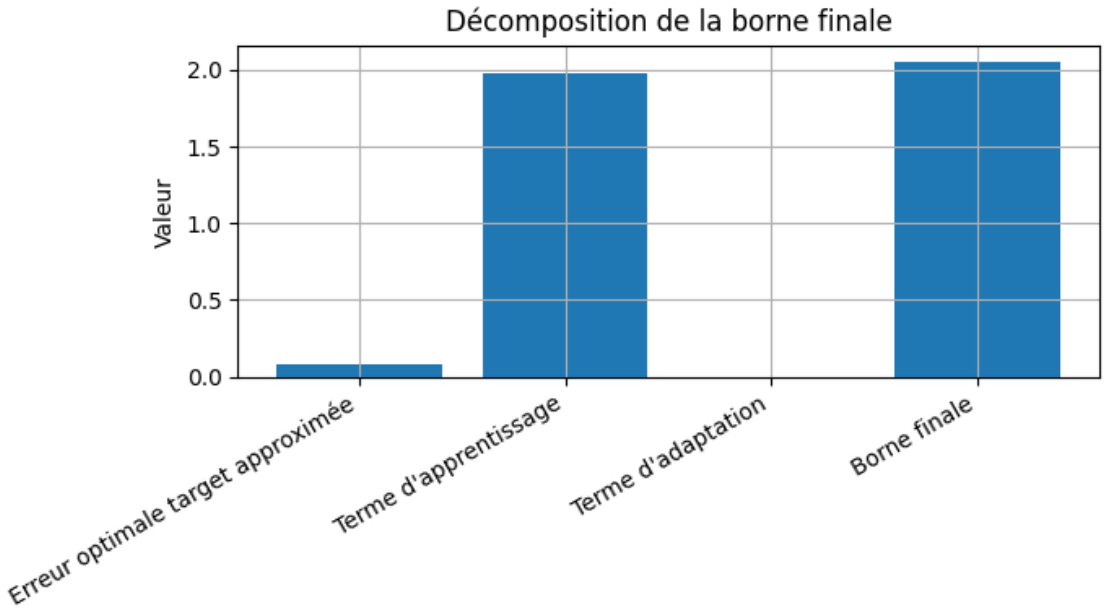
[53]:

| | Terme | Valeur |
|---|---|---|
| 0 | Erreur optimale target approximée | 0.083333 |
| 1 | Terme d'apprentissage | 1.973999 |
| 2 | Terme d'adaptation | 0.000000 |
| 3 | Borne finale | 2.057333 |

[54]:
```
plt.figure(figsize=(7, 4))

plt.bar(
    decomposition_theorem_3["Terme"],
    decomposition_theorem_3["Valeur"],
)

plt.xticks(rotation=30, ha="right")
plt.ylabel("Valeur")
plt.title("Décomposition de la borne finale")
plt.tight_layout()
plt.show()
```




### 7.11 Conclusion après le Théorème 3

Le Théorème 3 permet de passer d'une borne théorique à une borne exploitable à partir d'échantillons finis. Il introduit une erreur empirique pondérée qui combine les informations provenant du domaine source et du domaine cible.

L'idée principale est que l'on ne dispose jamais directement des vraies erreurs $\varepsilon_S$ et $\varepsilon_T$. On les estime à partir d'échantillons. Le Théorème 3 contrôle alors l'écart entre cette quantité empirique pondérée et l'erreur target réelle.

Ce résultat joue donc un rôle de transition entre la théorie et la pratique. Les théorèmes précédents expliquent quelles quantités contrôlent l'erreur cible ; le Théorème 3 indique comment ces quantités peuvent être approchées avec des données finies, tout en tenant compte d'un terme de concentration.

La leçon principale est la suivante : l'adaptation de domaine ne dépend pas seulement du choix d'un bon modèle, mais aussi de la manière dont on combine les données source et cible, et de la quantité d'information disponible dans chaque domaine.

## 8 Synthèse des trois grands théorèmes

Cette section résume la progression théorique du rapport.

---

### 8.1 Théorème 1 — Première décomposition de l'erreur cible

Forme simplifiée :

$$\varepsilon_T(h) \leq \varepsilon_S(h) + \text{écart de labels} + d_1(D_S, D_T).$$

Idée principale :

Le Théorème 1 décompose l'erreur cible en trois contributions :

- l'erreur du modèle sur le domaine source ;
- le changement de fonction d'étiquetage entre la source et la cible ;
- le changement de distribution entre $D_S$ et $D_T$.

Rôle dans le rapport :

Ce résultat donne une première borne théorique. Il explique pourquoi un modèle performant sur le domaine source peut échouer sur le domaine cible si les labels changent ou si les distributions sont trop différentes.

---

### 8.2 Théorème 2 — Borne fondamentale d'adaptation de domaine

Forme simplifiée :

$$\varepsilon_T(h) \leq \varepsilon_S(h) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) + \lambda.$$

Idée principale :

Le Théorème 2 remplace une distance directe entre distributions par une divergence adaptée à la classe d'hypothèses :

$$d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T).$$

Cette divergence mesure la variation des zones de désaccord entre hypothèses lorsqu'on passe du domaine source au domaine cible.

Rôle dans le rapport :

C'est le résultat central de l'adaptation de domaine. Il relie l'erreur cible à trois termes :

- la performance sur le domaine source ;
- la divergence entre les domaines ;
- le terme $\lambda$, qui mesure l'existence d'une bonne hypothèse commune aux deux domaines.

---

### 8.3 Théorème 3 — Passage aux échantillons finis

Forme simplifiée :

$$\varepsilon_T(h)$$

est contrôlée par une erreur empirique pondérée et un terme de concentration.

Idée principale :

Le Théorème 3 permet de passer d'une borne théorique, définie sur les distributions, à une borne exploitable à partir d'échantillons finis.

Rôle dans le rapport :

Il justifie l'étude expérimentale et l'utilisation d'erreurs empiriques pondérées dans le code.

---

### 8.4 Conclusion de la synthèse

Le Théorème 1 donne une première intuition sur les sources possibles de l'erreur cible.

Le Théorème 2 fournit la borne fondamentale d'adaptation de domaine.

Le Théorème 3 rend cette analyse compatible avec une situation statistique réelle, où l'on ne dispose que d'un nombre fini d'observations.

## 9 Conclusion générale

Nous avons construit une illustration complète de la borne d'adaptation de domaine.

Le point de départ est le fait qu'un modèle performant sur le domaine source n'est pas nécessairement performant sur le domaine target lorsque les distributions diffèrent.

Le théorème 2 montre que l'erreur target d'une hypothèse peut être contrôlée par quatre termes :

$$\epsilon_S(h), \qquad \frac{1}{2}d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T), \qquad \lambda, \qquad \text{un terme de concentration.}$$

Le théorème 3 étend cette idée au cas où l'on apprend avec un mélange de données source et target.

Le critère d'apprentissage devient :

$$\hat{\epsilon}_\alpha(h) = \alpha\hat{\epsilon}_T(h) + (1-\alpha)\hat{\epsilon}_S(h).$$

Le paramètre $\alpha$ permet d'arbitrer entre deux sources d'information :

- les données source, souvent plus nombreuses ;
- les données target, plus pertinentes mais parfois plus rares.

Dans notre expérience, on observe que la borne du théorème 3 est plus grande que l'erreur target empirique. C'est attendu : la borne est conservative, car elle doit être valable avec forte probabilité et pour une classe entière d'hypothèses.

L'intérêt principal de cette borne n'est donc pas de prédire exactement l'erreur observée, mais d'expliquer quels facteurs rendent l'adaptation possible :

$$\text{faible erreur source} + \text{faible divergence entre domaines} + \text{faible } \lambda + \text{suffisamment de données.}$$

Ainsi, l'adaptation de domaine est favorable lorsque les deux domaines sont proches et qu'il existe une hypothèse capable de bien fonctionner sur les deux.